

refine

Palabra clave correspondiente a una dependencia de refinamiento en la notación.

relación

Dícese de una conexión semántica materializada entre elementos de un modelo. Entre las clases de relaciones se cuentan la asociación, generalización, metarelación, flujo y distintos grupos que están agrupados con el nombre de dependencia.

Semántica

La Tabla 13.2 muestra las distintas clases de relaciones que hay en UML. La primera columna (clase) denota los agrupamientos en que se organizan en el metamodelo. La segunda columna

Tabla 13.2 Relaciones en UML

<i>Clase</i>	<i>Variedad</i>	<i>Notación</i>	<i>➤ Símbolo o palabra reservada</i>
abstracción	derivación	dependencia	« derive »
	realización	realización	----- ▷
	refinamiento	dependencia	« refine »
	traza	dependencia	« trace »
asociación		asociación	————
enlazado		dependencia	« bind » (parametros _{list,})
extensión		dependencia	« extend » (extensión puntos _{lista,})
flujo	conversión	dependencia	número-secuencia: « become »
	copia	dependencia	número-secuencia: « copy »
generalización		generalización	——▷
inclusión		dependencia	« include »
metarelación	instancia	dependencia	« instanceOf »
	supratipo	dependencia	« powertype »
permiso	acceso	dependencia	« access »
	amigo	dependencia	« friend »
	importación	dependencia	« import »
utilización	llamada	dependencia	« call »
	instanciación	dependencia	« instantiate »
	parámetro	dependencia	« parameter »
	envío	dependencia	« send »

(variedad) muestra las distintas clases de relaciones. La tercera columna (notación) muestra la notación base para cada relación: la asociación es una ruta continua, la dependencia es una flecha discontinua y la generalización es una ruta continua con punta de flecha triangular. La cuarta columna (palabra reservada) muestra las palabras reservadas de texto y la sintaxis adicional de aquellas relaciones que la requieren.

repositorio

Lugar de almacenamiento para modelos, interfaces e implementaciones; forma parte de un entorno para manipular artefactos de desarrollo.

requisito

Una característica, propiedad o comportamiento que se desea para el sistema.

Semántica

Un requisito de texto se puede modelar como un comando al que se adjunta el estereotipo «**requirement**».

Discusión

El término *requisito* es una palabra del lenguaje natural que se corresponde con una gama de estructuras de UML destinadas a especificar las características deseadas para un sistema. Los requisitos correspondientes a transacciones visibles por parte del usuario suelen capturarse como casos de uso. Los requisitos no funcionales, como las métricas de rendimiento y de calidad, se pueden capturar en forma de enunciados de texto que eventualmente llegarán a los elementos del diseño final. Los comentarios y restricciones de UML se pueden emplear para representar requisitos no funcionales.

responsabilidad

Contrato u obligación de una clase u otro elemento.

Semántica

Una responsabilidad puede representarse como un estereotipo en un comentario. El comentario se asocia a la clase u otro elemento que tenga la responsabilidad. Ésta se expresa en la forma de una cadena de texto.

Notación

Las responsabilidades se pueden mostrar en un compartimento con nombre situado dentro del rectángulo que es el símbolo de un clasificador (Figura 13.166).

LíneaCrédito	
adeudar (): Booleano abonar () ajustar (límite: Dinero, código: Cadena)	compartimento de operación predefinido
responsabilidades adeudar cargos pagar débitos rechazar cargos por encima del límite ajustar límites con autorización notificar a contabilidad si moroso llevar la cuenta del límite de crédito llevar la cuenta de cargos actuales	compartimento de responsabilidad definido por el usuario lista de responsabilidades

Figura 13.166 Compartimento de responsabilidades

restricción

Una condición semántica o limitación representada como expresión. Ciertas restricciones se predefinen en UML, otras se pueden definir por los modeladores. Las restricciones son uno de tres mecanismos de extensión en UML.

Véase también expresión, estereotipo, valor etiquetado.

Véase Capítulo 14, Elementos Estándar, para una lista de restricciones predefinidas.

Semántica

Una restricción es una condición o limitación semántica expresada como declaración lingüística en un determinado lenguaje textual.

En general, una restricción se puede unir a cualquier elemento o lista de elementos del modelo. Representa información semántica unida a un elemento del modelo, no sólo a una vista de él. Cada restricción tiene un cuerpo y un lenguaje de interpretación. El cuerpo es una cadena que codifica una expresión booleana para la condición en el lenguaje de la restricción. Una restricción se aplica a una lista ordenada de uno o más elementos del modelo. Observe que el lenguaje de especificación puede ser un lenguaje formal o puede ser lenguaje natural. En el último caso, la restricción será informal y no está sujeta a la ejecución automática (que no es decir que la ejecución automática sea siempre práctica para todos los lenguajes formales). UML proporciona el lenguaje de restricciones OCL [Warner-99], pero es posible usar otros lenguajes.

Algunas restricciones comunes tienen nombres para evitar escribir una declaración completa cada vez que son necesarias. Por ejemplo, la restricción **xor** entre dos asociaciones que compartan una clase común significa que un solo objeto de la clase compartida puede pertenecer a solamente una de las asociaciones al mismo tiempo.

Véase Capítulo 14, Elementos Estándar, para una lista de las restricciones predefinidas de UML.

Una restricción es una aserción, no un mecanismo ejecutable. Indica una condición que se debe hacer cumplir para el correcto diseño del sistema. Cómo garantizar una restricción es una decisión de diseño. Las restricciones de ejecución se hacen para ser evaluadas en los momentos en que el sistema instanciado es estable —es decir, entre la ejecución de operaciones y no en el medio de cualquier transacción atómica—. Durante la ejecución de una operación, puede haber momentos en que las restricciones se violen temporalmente.

Una restricción no se puede aplicar a sí misma.

Una restricción heredada —una restricción en un elemento antecesor del modelos o un estereotipo— debe seguirse aunque se definan restricciones adicionales en los descendientes. Una restricción heredada no puede ser pasada por alto o reemplazada. Si necesita hacerlo, el modelo está mal construido y debe ser reformulado. Sin embargo una restricción heredada se puede restringir, agregando restricciones adicionales. Si están en conflicto con las restricciones heredados por un elemento, entonces el modelo está mal formado.

Notación

Una restricción se representa como una cadena de texto entre llaves ({ }). La cadena de texto es el cuerpo codificado escrito en un lenguaje de restricción.

Es de esperar que las herramientas proporcionen uno o más lenguajes en los cuales se puedan escribir las restricciones formales. El lenguaje predefinido para la escritura de restricciones es OCL. Dependiendo del modelo, un lenguaje de programación tal como C++ puede ser útil para algunas restricciones. Si no, la restricción se puede escribir en lenguaje natural, con responsabilidad humana cara a la interpretación y ejecución. El lenguaje de cada restricción es parte de la restricción en sí misma, aunque el lenguaje no se expone generalmente en el diagrama (la herramienta no lo pierde de vista).

Para una lista de los elementos representados por las cadenas de texto en un compartimento (tal como los atributos dentro de una clase): una cadena de restricción puede aparecer como una entrada en la lista (Figura 13.167). La entrada no representa un elemento del modelo. Es una *restricción viva* que se aplica a todos los elementos correctos de la lista hasta otro elemento de la lista de restricciones o el fin de la lista. La restricción viva se puede sustituir por otra restricción viva más adelante en la lista. Para eliminar una restricción de ejecución, la sustituimos por una restricción vacía. Una restricción unida a un elemento individual de la lista no sustituye la restricción viva, pero puede aumentarla con restricciones adicionales.

Para un solo símbolo gráfico (tal como una clase o una línea de asociación), la cadena de restricción se puede colocar cerca del símbolo, preferiblemente cerca del nombre del símbolo, si lo hay.

Transacción CA	
{ valor $\geq$ 0 }	restricción de ejecución
cantidad: Dinero { valor es múltiplo de 20\$ }	restricción individual y restricción viva
saldo: Dinero	sólo se aplica la restricción de ejecución

Figura 13.167 Restricciones dentro de listas

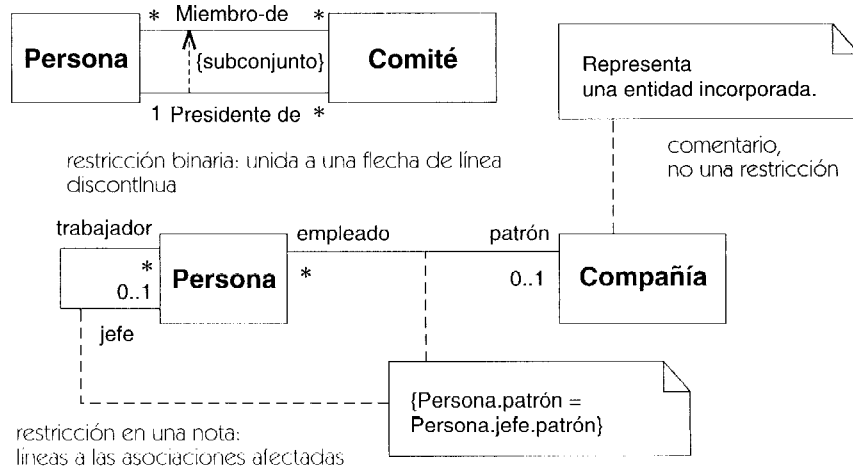


Figura 13.168 Notación de restricciones

Para dos símbolos gráficos (tales como dos clases o dos asociaciones): la restricción se representa como una flecha con línea discontinua de un elemento al otro elemento etiquetado por la cadena de la restricción (entre llaves). La dirección de la flecha es información relevante dentro de la restricción (Figura 13.168).

Para tres o más símbolos gráficos: La cadena de la restricción se coloca en un símbolo de nota y es unida a cada símbolo por una línea discontinua (Figura 13.168). Esta notación se puede utilizar para otros casos también. Para tres o más líneas del mismo tipo (tales como líneas de generalización o líneas de asociación), la restricción se puede unir a una línea discontinua que atraviesa todas las líneas. En caso de ambigüedad, las diferentes líneas se pueden numerar o etiquetar para establecer su correspondencia con la restricción.

Discusión

Una restricción hace una declaración semántica sobre el modelo en sí mismo, mientras que un comentario es un texto sin fuerza semántica y se puede unir a un elemento del modelo o a un elemento de representación. Ambos, restricciones y comentarios se pueden representar usando notas. En principio, las restricciones son ejecutables por las herramientas. En la práctica, algunas pueden ser difíciles de indicar formalmente y pueden requerir la ejecución humana. En el amplio sentido de la palabra, muchos elementos en un modelo son restricciones, pero la palabra restricción se utiliza para indicar las declaraciones semánticas que son difíciles de expresar con los elementos incorporados al modelo y que se deben indicar de manera lingüística.

Las restricciones se pueden expresar en cualquier lenguaje conveniente o aun en un lenguaje humano, aunque una restricción en lenguaje humano no se puede verificar por una herramienta.

El lenguaje OCL [Warmer-99] está diseñado para especificar restricciones de UML, pero bajo algunas circunstancias puede ser más apropiado un lenguaje de programación.

Ya que las restricciones se expresan como cadenas de texto, una herramienta de modelado genérica puede admitirlas y mantener su significado sin entenderlo. Por supuesto, una herra-

mienta o un dispositivo suplementario que verifique o haga cumplir la restricción debe entender la sintaxis y la semántica del lenguaje.

Se puede unir una lista de restricciones a la definición de un estereotipo. Esto indica que todos los elementos que llevan el estereotipo están conforme a la restricción.

Ejecución. Cuando el modelo contiene restricciones, no es necesario especificar qué hacer si son violadas. Un modelo es una declaración de lo que se supone que debe suceder. El hacer que suceda es trabajo de la implementación. Un programa puede contener aserciones y otros mecanismos de la validación, pero una falta contra una restricción se debe considerar una falta de programación. Por supuesto, si un modelo puede ayudar a producir un programa que esté listo para la construcción o se pueda verificar como correcto, entonces ha respondido a su propósito.

Elementos estándar

invariant, postcondition, precondition.

reunificación

Un lugar de una máquina de estados, diagrama de actividades o diagrama de secuencia en el cual se unen dos o mas rutas de control alternativas; una “desbifurcación”. Antónimo: bifurcación.

Véase también estado de unión o conjunción.

Semántica

Una reunificación no es otra cosa que una situación en la que se juntan dos o más rutas de control. En una máquina de estados, un estado tiene más de una transición de entrada. No se requiere ni se proporciona ninguna estructura especial del modelo para indicar una reunificación. Se puede indicar mediante un estado de unión si forma parte de una única ruta que se ejecuta hasta finalizar.

Notación

Una reunificación puede aparecer en un diagrama de estados, un diagrama de actividades o un diagrama de secuencia como un rombo con dos o más transiciones de entrada y una sola transición de salida. No se necesitan condiciones. La Figura 13.169 muestra un ejemplo.

También se emplea un rombo para las bifurcaciones (que son la inversa de las fusiones), pero la bifurcación se distingue claramente porque posee una transición de entrada y múltiples transiciones de salida, cada una de las cuales posee su propia condición de guarda.

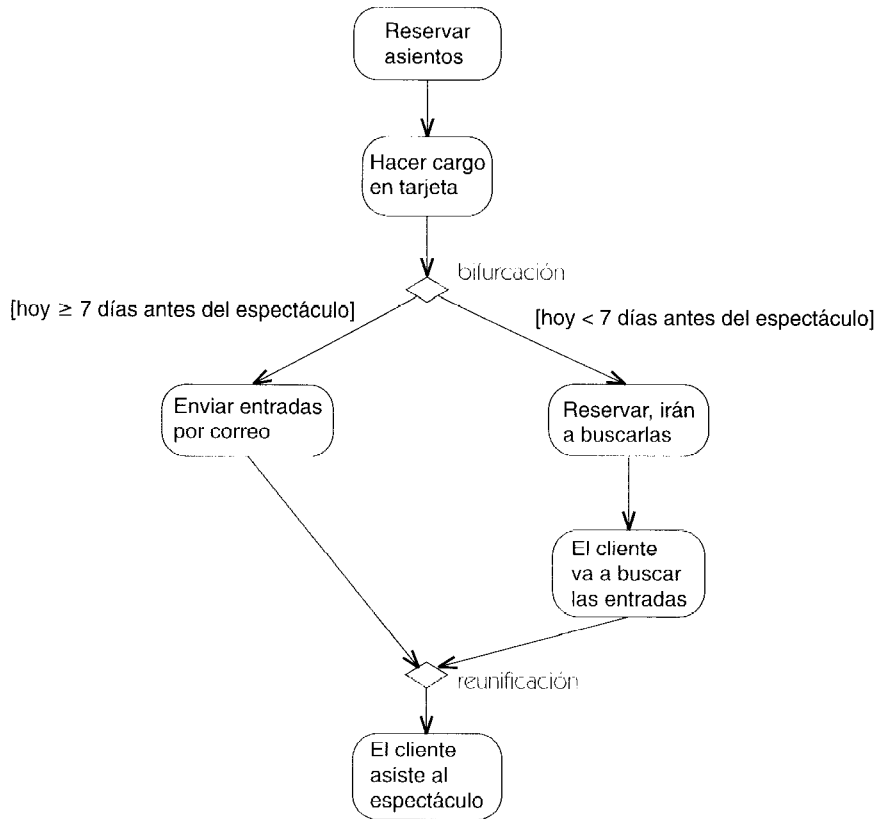


Figura 13.169 Reunificación

Una combinación de bifurcación y reunificación es válida pero tiene una utilidad limitada. Tendría múltiples transiciones de entrada y múltiples transiciones de salida con etiqueta.

Obsérvese que una reunificación es tan sólo una forma cómoda de notación y se puede omitir sin pérdida de información. Normalmente, las bifurcaciones y las fusiones se emparejan de forma anidada.

Discusión

Asegúrese de que distinga las reunificaciones de las uniones. Las reunificaciones combinan dos o más rutas alternativas de control. En toda ejecución se seguirá solamente una ruta en cada momento. No se necesita una sincronización.

La unión combina dos o más rutas concurrentes de control. En toda ejecución se seguirán todas las rutas y la unión sólo se disparará cuando todas ellas hayan llegado a los estados de origen de la unión.

reutilización

Utilización de un artefacto existente previamente.

rol

Ranura con nombre situada dentro de la estructura de un objeto que representa el comportamiento de un elemento que participa en un contexto (por oposición a las cualidades inherentes del elemento en cualquiera de sus aplicaciones). Un rol puede ser estático (como un extremo de asociación) o dinámico (como un rol de colaboración). Entre los roles de colaboración se cuentan los roles de clasificador y los roles de asociación.

Véase colaboración.

rol de asociación

Conexión de dos roles de clasificador dentro de una colaboración. Asociación entre dos clasificadores que se aplica sólo a un cierto contexto especificado por una colaboración.

Véase también asociación, colaboración.

Semántica

Un rol de asociación es una asociación que está definida y tiene significado sólo en el contexto descrito por una colaboración. Es una relación que es parte de la colaboración pero no una relación inherente en otras situaciones. Los roles de asociación son la parte estructural clave de las colaboraciones, pues permiten describir relaciones contextuales.

Dentro de una colaboración, un rol de clasificador denota una aparición individual de un clasificador, distinta de otras apariciones del clasificador y distinta también de la propia declaración del clasificador. Es un clasificador por derecho propio que representa una restricción en el uso del clasificador base basada en el contexto de una colaboración. De forma similar, un rol de asociación representa una asociación utilizada en un contexto particular, a menudo un uso restringido de una asociación normal. Un rol de asociación conecta dos roles de clasificador. Cuando se instancia una colaboración, los objetos se enlazan a roles de clasificador y los enlaces a roles de asociación. Un objeto puede desempeñar (enlazarse a) más de un rol.

Un rol de asociación conecta dos o más clasificadores o roles de clasificador dentro de una colaboración. El rol de asociación tiene una referencia a la asociación —la asociación base— y puede tener una multiplicidad que indique cuántos enlaces pueden representar el rol en una instancia de la colaboración. En algunos casos, las conexiones dentro de la colaboración pueden verse como usos de una asociación general entre las clases participantes. La colaboración muestra una forma de utilizar la asociación general para un propósito determinado dentro de la colaboración.

En otros casos, los roles de clasificador están conectados mediante asociaciones que no tienen validez fuera de la colaboración. Si un rol de asociación no tiene una asociación base explícita, entonces define una asociación implícita válida únicamente dentro de la colaboración.

Notación

Un rol de asociación se representa de la misma forma que una asociación, en concreto, mediante una línea continua entre dos símbolos de rol de clasificador (véase Figura 13.170). Es fácil distinguir que se trata de un rol de asociación pues conecta roles de clasificador.

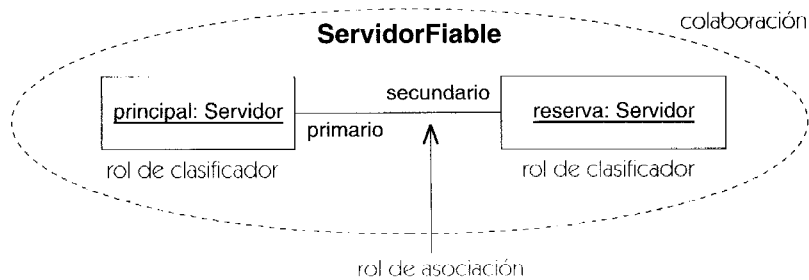


Figura 13.170 Rol de asociación

Elementos estándar

new, transient.

rol de clasificador

Ranura de una colaboración que describe el rol desempeñado por un participante en una colaboración.

Véase también colaboración.

Semántica

Una colaboración describe un patrón de interacción entre un conjunto de participantes, que son instancias de clases o de tipos de datos. Un rol de clasificador es la descripción de un participante. Cada rol es un uso distinto del clasificador en un contexto propio y único. Puede haber más de un rol para el mismo clasificador, cada uno de los cuales tendrá un conjunto distinto de relaciones con otros roles dentro de una colaboración. Sin embargo, un rol no es un objeto individual, sino una descripción de todos los objetos que pueden tomar parte en una instancia de una colaboración. En cada instancia de la colaboración, los roles serán desempeñados por diferentes objetos y enlaces.

Un rol de clasificador tiene una referencia a un clasificador (clasificador base) y una multiplicidad. El clasificador base restringe el tipo de objeto que puede desempeñar el rol de clasificador. La clase del objeto puede ser la misma o un descendiente del clasificador base. La multiplicidad indica cuántos objetos pueden desempeñar el rol a la vez en una instancia de la colaboración.

Un rol de clasificador puede tener un nombre o ser anónimo. Puede tener múltiples clasificadores base si se realiza clasificación múltiple.

Un rol de clasificador puede conectarse con otros roles de clasificador mediante roles de asociación.

Objetos. Una colaboración representa un grupo de objetos que trabajan juntos para llevar a cabo un objetivo. Un rol representa la parte de uno de los objetos (o un grupo de los objetos) que reali-

za dicho objetivo. Un objeto es una instancia directa o indirecta de la clase base de su rol. No todos los objetos de la clase base aparecen necesariamente en una colaboración de su rol, además es posible que objetos de la misma clase base desempeñen múltiples roles en la misma colaboración.

El mismo objeto puede desempeñar diferentes roles en diferentes colaboraciones. Una colaboración representa una faceta de un objeto, pudiendo combinar un mismo objeto físico diferentes facetas conectando implícitamente las colaboraciones en las que desempeña algún rol.

Notación

Un rol de clasificador se representa utilizando el símbolo de clasificador (un rectángulo) con un nombre de rol y de clasificador separados por dos puntos, esto es, nombreRol:ClaseBase. Sin embargo, un rol no es un objeto, sino un clasificador que describe muchos objetos que aparecen en diferentes instancias de la colaboración.

Se puede omitir el nombre del rol o el del clasificador, pero deben aparecer los dos puntos para distinguirlo de una clase ordinaria. Dentro de una colaboración existe un pequeño riesgo de confusión, pues todos los participantes son roles.

Un rol de clasificador puede mostrar un subconjunto de las características del clasificador, es decir, los atributos y operaciones utilizados en un contexto dado. El resto de características pueden suprimirse si no se utilizan.

La Figura 13.171 muestra diversas formas que puede tomar un rol de clasificador.

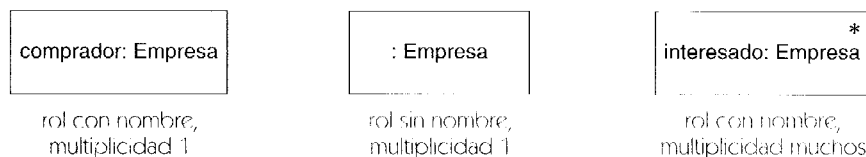


Figura 13.171 Rol de clasificador

Elementos estándar

destroyed, new, transient.

rol de colaboración

Ranura para un objeto o enlace dentro de una colaboración. Especifica el tipo de objeto o enlace que puede aparecer en una instancia de la colaboración.

Véase también colaboración, rol de asociación, rol de clasificador.

Semántica

Un rol de colaboración describe un objeto o un enlace. Sin embargo, no representa un único enlace u objeto sino un sitio que puede sustituirse por un objeto o un enlace cuando se

instancia la colaboración. Una colaboración es o bien un rol de clasificador o bien un rol de asociación. Un rol de clasificador tiene uno o más clasificadores base y puede ser instanciado como un objeto, el cual es una instancia de los clasificadores o uno de sus descendientes. Un rol de asociación puede tener una asociación base y ser instanciado como un enlace, el cual es una instancia de la asociación o uno de sus descendientes. En muchos casos, la asociación sólo está definida en la colaboración —esto es, ocurre sólo entre objetos que desempeñan los roles y no tiene significado fuera de la colaboración—. En estos casos se puede omitir la asociación base. La asociación se define implícitamente por su aparición en la colaboración, con la restricción de que no puede ser utilizada en otro sitio.

Notación

Los roles de colaboración pueden ser roles de clasificador o roles de asociación.

Rol de clasificador. Un rol de clasificador es un clasificador y se representa mediante el símbolo de las clases: un rectángulo. A menudo sólo se muestra el compartimento del nombre, que contiene la cadena:

```
nombreRolClasificador :NombreClasificador Baseinst.
```

El nombre de clasificador base puede incluir la ruta de paquetes completa si es necesario (una herramienta permitirá normalmente caminos abreviados cuando no haya ambigüedad). Los nombres de paquete preceden al nombre de la clase separados por dos signos de dos puntos, por ejemplo:

```
mostrar_ventana:SistemaVentanas::VentanasGraficas:: Ventana
```

Un estereotipo para un rol de clasificador puede representarse textualmente entre comillas dobles sobre la cadena del nombre o como un icono en la esquina superior derecha. El estereotipo de un rol de clasificador debe coincidir con el estereotipo de su clasificador base.

Un rol de clasificador que representa un conjunto de objetos incluye un indicador de multiplicidad (por ejemplo un asterisco) en la esquina superior derecha del símbolo de la clase. Este indicador especifica el número de objetos que pueden ligarse al rol en una instancia de la colaboración. Cuando se omite el indicador, el valor es exactamente uno.

El nombre del rol de clasificador puede omitirse, en cuyo caso deberían mantenerse los dos puntos al lado del nombre de la clase. Este caso representa un objeto anónimo de la clase.

Si hay clasificación múltiple, el rol puede tener más de un clasificador, en ese caso, el objeto es una instancia de cada uno de ellos.

La clase del rol de clasificador puede suprimirse (junto con los dos puntos).

Un objeto o enlace que se crea durante una interacción tiene la palabra clave **new** como restricción en su rol. Un objeto o enlace destruido durante una interacción tiene la palabra clave **destroyed** como restricción en su rol. Las palabras clave pueden utilizarse incluso aunque el elemento no tenga nombre. Ambas palabras clave pueden utilizarse conjuntamente, pero resulta más útil emplear la palabra clave **transient** en lugar de la combinación **new destroyed**.

Rol de asociación. Un rol de asociación es una asociación, y se representa mediante una ruta entre dos símbolos de rol de clasificador. La ruta puede tener asociado un nombre de la forma:

```
nombreRolAsociación NombreAsociaciónBase
```

Si se omite el nombre de la asociación base, entonces no hay asociación base. En la ruta pueden mostrarse los nombres de rol y otros adornos de la asociación base.

Si un extremo de la ruta del rol de asociación está conectado a un rol de clase con una multiplicidad distinta de uno, puede situarse un indicador de multiplicidad en ese extremo de la asociación para enfatizar dicha multiplicidad.

La Figura 13.46 muestra un ejemplo de roles de colaboración.

ruta

Se trata de una serie conexas de segmentos gráficos que conectan símbolos entre sí, normalmente para mostrar una relación.

Notación

Las *rutas* son conexiones gráficas entre los símbolos de un diagrama. Las rutas se utilizan en esta notación para las relaciones de distintas clases, tales como asociaciones, generalizaciones y dependencias. Los extremos de dos segmentos conectados coinciden. Un segmento puede ser un segmento de línea recta, un arco o alguna otra forma, aun cuando muchas herramientas admiten únicamente líneas rectas y arcos (Figura 13.172). Las líneas se pueden dibujar en cualquier ángulo aunque algunos creadores de modelos prefieren limitar las líneas a ángulos rectos y posiblemente impongan una cuadrícula rectangular por razones estéticas y para facilitar el trazado. En general, el trazado de una ruta no tiene significado, aunque las rutas deberían evitar cruzar regiones cerradas, porque cruzar los límites de una región gráfica puede tener significado semántico. (Por ejemplo, una asociación entre dos clases de una colaboración debería dibujarse dentro de la región de colaboración para indicar una asociación entre objetos de la misma instancia de colaboración; sin embargo, una ruta que hiciera una salida de la región indicaría una asociación entre objetos procedentes de distintas instancias de colaboración.) Más exactamente, las rutas son topológicas. Su trazado exacto no tiene semántica, pero su conexión con otros símbolos y sus intersecciones poseen un significado. El trazado exacto de las rutas tiene gran importancia a efectos de la compresión y de la estética, por supuesto; puede connotar de forma sutil la importancia de las relaciones y otras cosas. Pero estas consideraciones son para los seres humanos y no para los computadores. Se supone que las herramientas admitirán de forma sencilla los trazados y retrazados de las rutas.

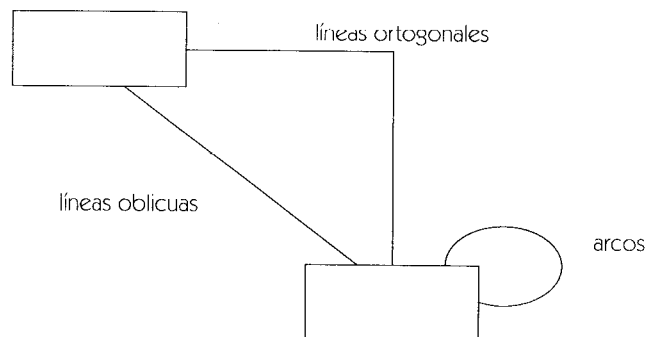
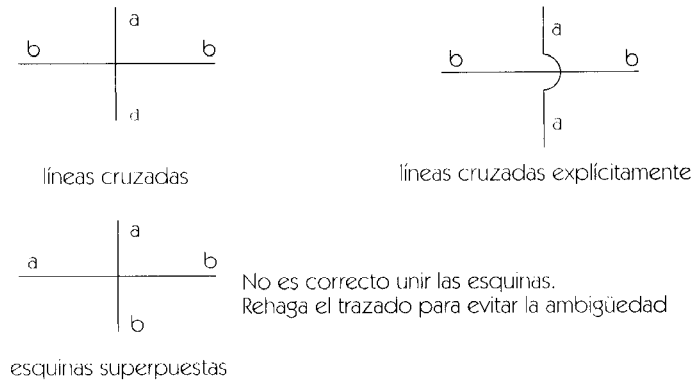
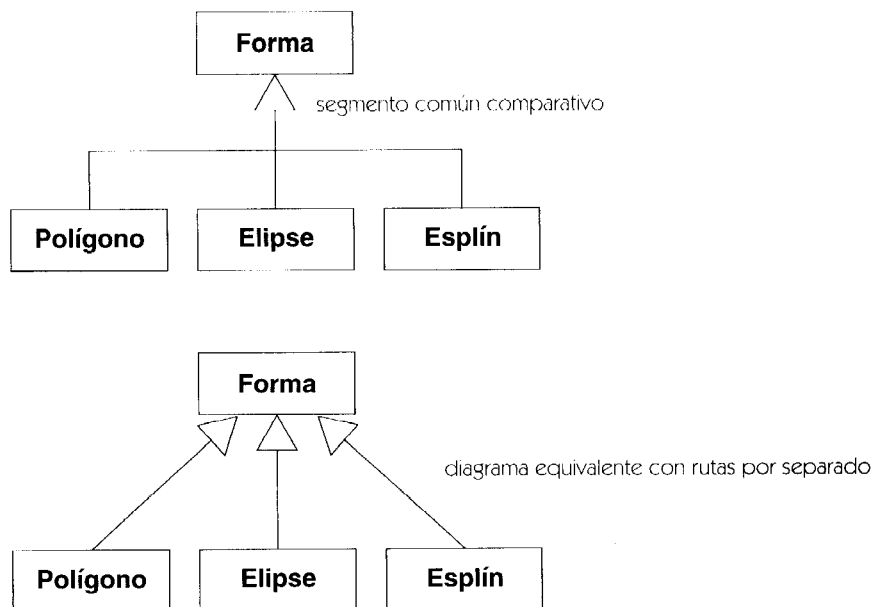


Figura 13.172 Rutas

**Figura 13.173** Cruces de rutas

En la mayoría de los diagramas, los cruces de líneas carecen de significación. Para evitar la ambigüedad respecto a la identidad de las líneas que se cruzan, se puede dibujar un pequeño semicírculo o discontinuidad en una de las líneas en el punto de cruce (Figura 13.173). Con mayor frecuencia, los creadores de modelos se limitan a tratar el cruce como dos líneas independientes y están de acuerdo en evitar la confusión que surgiría al hacer que dos ángulos rectos se tocaran en el vértice.

En algunas relaciones (tales como la agregación y la generalización), varias rutas del mismo tipo se pueden conectar a un mismo símbolo. Si las propiedades de los distintos elementos del modelo coinciden, entonces los segmentos de línea conectados con el símbolo se pueden combinar en un único segmento de línea, de tal modo que la ruta que sale del símbolo se ramifique formando una especie de árbol (Figura 13.174). Esto no pasa de ser una opción de

**Figura 13.174** Rutas con segmento compartido

representación gráfica. Conceptualmente, las rutas individuales son diferentes. Esta opción de representación no puede utilizarse cuando la información de modelado de los diferentes segmentos no es idéntica.

secuencia de acciones

Sucesión de acciones que se ejecutan una detrás de otra. Es un tipo de acción.

Semántica

Una secuencia de acciones es una lista de acciones. Las acciones se ejecutan secuencialmente. La secuencia completa se considera una unidad atómica (es decir, no puede ser interrumpida). Una secuencia de acciones es una acción y puede por consiguiente ser asociada a transiciones o a pasos de interacción.

Notación

Una secuencia de acciones se representa mediante una cadena formada por una secuencia de cadenas separadas por signos de punto y coma.

Si las acciones se expresan en un determinado lenguaje de programación, se puede utilizar la sintaxis sobre secuencia de acciones del lenguaje en lugar de la representación descrita.

Ejemplo

```
contador:=0;reservas.borrar();send Quiosco.primeraPantalla()
```

semántica

Especificación formal del significado y comportamiento de algo.

señal

Especificación de una comunicación asíncrona entre objetos. Las señales pueden tener parámetros que se expresan como atributos.

Véase también evento, mensaje, enviar.

Semántica

Una señal es un clasificador explícito con nombre, destinado a una comunicación explícita entre objetos. Posee una lista explícita de parámetros, representados como sus atributos. Es enviada

explícitamente por un objeto a otro objeto o conjunto de objetos. La emisión general de una señal se puede considerar como enviar una señal al conjunto de todos los objetos —aunque en la práctica se implementaría de otra manera para mayor eficiencia—. El emisor especifica los argumentos de la señal en el momento en que se envía ésta. Enviar una señal es equivalente a instanciar un objeto señal y transmitir ese objeto al conjunto de objetos destino. La recepción de una señal es un evento que está destinado a disparar transiciones en la máquina de estados del receptor. Una señal que se envía a un conjunto de objetos puede disparar cero o una transiciones en cada objeto independientemente. Las señales son medios explícitos mediante los cuales los objetos se pueden comunicar entre sí asíncronamente. Para efectuar una comunicación síncrona, es preciso utilizar dos señales, una en cada dirección.

Las señales son elementos generalizables. Una señal hija se deriva de una señal padre. Hereda los atributos del padre y puede agregar sus propios atributos adicionales. Una señal hija dispara una transición que haya sido declarada para utilizar una de sus señales antecesoras.

La declaración de una señal tiene como alcance el paquete en que haya sido declarada. No está restringida a una única clase.

Una clase o una interfaz pueden declarar las señales que están dispuestas a manejar. Esta declaración es una recepción, que puede incluir la especificación de los resultados que se esperan cuando se reciba la señal. La declaración posee una propiedad que indica si es polimórfica. Si lo es, entonces una clase descendiente podrá manejar la señal, evitando quizá que la señal llegue a la clase actual. Si no es polimórfica, entonces ningún descendiente podrá interceptar el manejo de la señal.

Una señal es un clasificador y puede tener operaciones que accedan a sus atributos y los modifiquen. Además todas las señales comparten la operación implícita `send`.

`send (conjuntoDestino)`

La señal se envía a todos los objetos del conjunto destino.

Notación

La palabra reservada de estereotipo «**signal**» se pone delante de la declaración de aquellos parámetros que tengan el nombre de una señal, para indicar que hay una clase o interfaz que admite la señal. Los parámetros de la señal se incluyen en la declaración. La declaración no puede tener un tipo de retorno.

La declaración de una señal se puede expresar como un estereotipo de un símbolo de clase. La palabra reservada «**signal**» aparece en un rectángulo por encima del nombre de la señal. Los parámetros de la señal aparecen como atributos dentro del compartimento de atributos. El compartimento de operaciones puede contener operaciones de acceso.

La Figura 13.175 muestra la utilización de una notación de generalización para relacionar una señal hija con su padre. La hija hereda los parámetros de sus antecesores y puede agregar sus propios parámetros adicionales. Por ejemplo, la señal **BotonRatonPulsado** tiene los atributos **tiempo**, **dispositivo** y **localización**.

Para utilizar una señal como disparador de una transición, utilice la sintaxis:

`nombre-evento ( parámetroslista )`

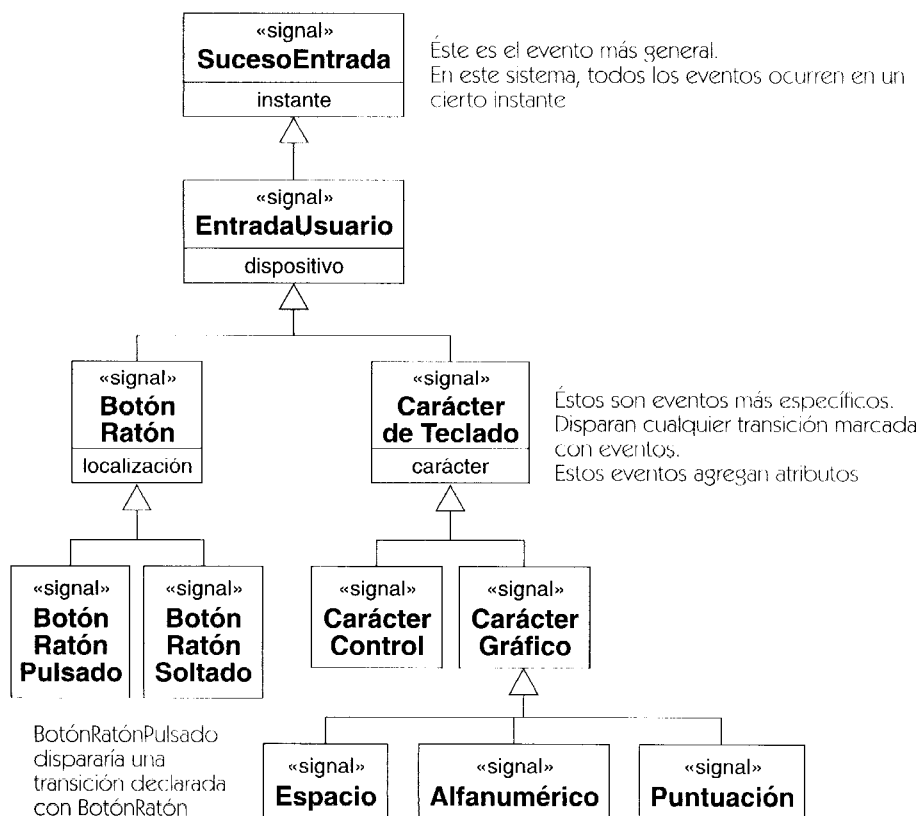


Figura 13.175 Declaraciones de señal

Los parámetros tienen la sintaxis:

nombre-parámetro :expresión-tipo

Los parámetros de una señal se declaran como atributos con valor inicial, que se puede anular durante la iniciación o al enviar. El valor inicial se usa si se crea una instancia de señal, se inicia y después se envía a un objeto. Si se envía una señal empleando una sintaxis de llamada a operación, entonces los valores iniciales son los valores por defecto de los parámetros de la señal.

Discusión

Una señal es la más fundamental comunicación entre objetos; posee una semántica más sencilla y más clara que las llamadas a procedimientos. Una señal es inherentemente a una comunicación asíncrona unidireccional que va de un objeto a otro y en la cual se pasa por valor toda la información. Es un modelo adecuado para la comunicación en sistemas concurrentes y distribuidos.

Para construir una comunicación asíncrona, utilice parejas de señales (una en cada dirección). Una llamada se puede ver como una señal con un parámetro implícito que es un puntero de retorno.

signatura

Nombre y propiedades de los parámetros de una característica de comportamiento, tal como una señal u operación. La signatura puede incluir tipos de retorno opcionales (para las operaciones, no para las señales).

Semántica

La signatura de una operación forma parte de su declaración. Parte de la signatura (pero no toda) se emplea para verificar la equivalencia de operaciones y métodos, con objeto de evitar conflictos o anulaciones. Para este propósito, se incluye el nombre de la operación y la lista ordenada de los tipos de los parámetros, pero no sus nombres ni direcciones y se excluyen los parámetros de retorno. Si coinciden dos signaturas pero las propiedades restantes son inconsistentes (por ejemplo, si un parámetro de entrada corresponde a uno de salida) entonces las declaraciones están en conflicto y el modelo está mal formado.

sistema

Colección de unidades conectadas que se organiza para lograr un propósito. Un sistema puede estar descrito por uno o más modelos, posiblemente desde distintos puntos de vista. El sistema es el “modelo completo”.

Semántica

El sistema se modela mediante un subsistema de nivel máximo que contiene indirectamente todo el conjunto de elementos del modelo que funcionan como un todo para lograr algún propósito del mundo real.

Notación

Se puede mostrar el sistema como un paquete con el estereotipo «**system**». Sin embargo, no suele ser necesario mostrar el sistema como una unidad.

solicitud

Especificación de un estímulo que se envía a las instancias. Puede ser la llamada a una operación o bien el envío de una señal.

subclase

Hija de otra clase en una relación de generalización —esto es, la descripción más específica—. La clase hija se denomina subclase. La clase padre se denomina superclase.

Véase generalización, herencia.

Semántica

Una subclase hereda la estructura, relaciones y comportamiento de su superclase; además puede hacer sus propias adiciones.

subestado

Estado que forma parte de un estado compuesto.

Véase estado compuesto, subestado concurrente, subestado disjunto.

subestado concurrente

Un subestado que se puede llevar a cabo simultáneamente con otros subestados contenidos en el mismo estado compuesto.

Véase estado compuesto, subestado disjunto.

subestado disjunto

Un subestado que no se puede llevar a cabo simultáneamente con otros subestados contenidos en el mismo estado compuesto. Contraste: subestado concurrente.

Véase transición compleja, estado compuesto.

subestado ortogonal

Se trata de uno elemento de un conjunto de estados que descomponen un estado compuesto en subestados, todos los cuales están activados concurrentemente.

Véase estado compuesto, subestado concurrente.

submáquina

Máquina de estados que se puede invocar como parte de otra máquina de estados. Posee una semántica tal como si se hubiera duplicado su contenido para luego insertarlo en el estado al que hace referencia.

Véase estado, máquina de estados, estado de referencia a submáquina.

subsistema

Paquete de elementos que se tratan como una unidad, incluyendo una especificación de todo el contenido del paquete, que se trata como una unidad coherente. Un subsistema se modela

simultáneamente como paquete y como clase. Los subsistemas tienen un conjunto de interfaces que describen su relación con el resto del sistema y las circunstancias en que se puede utilizar.

Véase también interfaz, paquete, realización.

Semántica

Un subsistema es una pieza coherente de un sistema que se puede tratar como unidad abstracta. Representa el comportamiento emergente de una parte del sistema. Como unidad, posee su propia especificación de comportamiento y su parte de implementación. La especificación de comportamiento define su comportamiento emergente como unidad que puede interactuar con otros subsistemas. La especificación de comportamiento se da en términos de caso de uso y otros elementos de comportamiento. La parte de implementación describe la implementación del comportamiento en términos de los elementos subordinados de que consta su contenido y se da como un conjunto de colaboraciones entre los elementos contenidos.

El sistema en sí constituye el subsistema de más alto nivel. La implementación de un subsistema se puede escribir como una colaboración de subsistemas de nivel inferior. De esta manera, se puede expandir todo el sistema como un jerarquía de subsistemas hasta definir los subsistemas del más bajo nivel en términos de clases ordinarias.

Los subsistemas pueden incluir elementos estructurales y elementos de especificación, tales como casos de uso y operaciones exportadas por el subsistema. Las especificaciones del subsistema se implementan mediante elementos estructurales. El comportamiento del subsistema es el comportamiento de los elementos que contiene.

Estructura

La especificación de un subsistema abarca los elementos indicados como elementos de especificación, junto con las operaciones e interfaces que se hayan definido para el subsistema como un todo. Entre los elementos de especificación se cuentan los casos de uso, las restricciones, las relaciones entre casos de uso y demás. Estos elementos y operaciones definen el comportamiento del subsistema como entidad emergente, que es el resultado neto de la actuación conjunta de sus partes. Los casos de uso especifican secuencias completas de interacciones del subsistema con actores externos. Las interfaces especifican operaciones que deban proporcionar el subsistema o sus casos de uso. La especificación no revela cuáles son las partes ni cómo interactúan esas partes para realizar el comportamiento necesario.

El resto de los elementos del subsistema se encarga de dar vida a su comportamiento. Esto puede incluir distintas clases de clasificadores y las relaciones existentes entre ellos. Hay un conjunto de colaboraciones entre elementos de realización del subsistema que se encarga de dar vida a las especificaciones. En general, cada caso de uso requiere una o más colaboraciones. Cada colaboración describe la forma en que cooperan las instancias de los elementos de implementación para realizar conjuntamente el comportamiento especificado por un caso de uso u operación. Todos los mensajes entrantes y salientes en el subsistema en el nivel de especificación tienen que corresponderse con mensajes entre sus elementos de implementación y los de otros subsistemas.

Un subsistema es un paquete y tiene las propiedades de un paquete. En particular, la importación de subsistemas funciona tal como se ha descrito para los paquetes y la generalización entre subsistemas tiene las mismas consecuencias para la visibilidad de su contenido.

Notación

Los subsistemas se denotan con un símbolo de paquete (un rectángulo con una pequeña ficha) que contiene la palabra reservada «**subsystem**» por encima del nombre del subsistema (Figura 13.176).

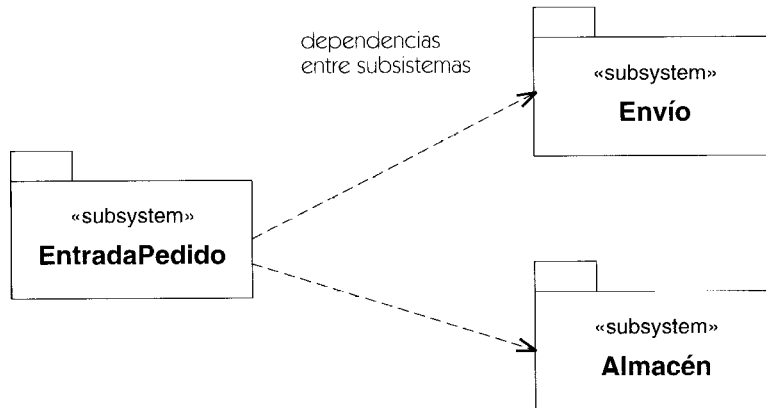


Figura 13.176 Subsistemas

Discusión

Un subsistema es un grupo emergente de elementos de diseño, tales como clases lógicas. Un componente es un grupo emergente de elementos de implementación, tales como clases del nivel de implementación. En muchos casos, los subsistemas se implementan como componentes. Esto simplifica la correspondencia entre diseño e implementación; por tanto, se trata de un enfoque corriente para la arquitectura. Además, muchos componentes se implementan como clases dominantes que implementan directamente las interfaces de los componentes. En este caso, un subsistema, un componente y una clase podrían tener todos ellos la misma interfaz.

subtipo

Tipo que es hijo de otro tipo. Se puede utilizar el término hijo, más neutro, para cualquier elemento generalizable

Véase generalización.

superclase

Padre de otra clase en una relación de generalización, esto es, la especificación de elemento más general. La clase hija se denomina subclase. La clase padre se denomina superclase.

Véase generalización.

Semántica

Las subclases heredan la estructura, relaciones y comportamiento de sus superclases y pueden hacer adiciones.

supertipo

Sinónimo de superclase. Se puede emplear el término más neutro, padre, para cualquier elemento generalizable.

Véase generalización.

supratipo

Denota una metaclass cuyas instancias son subclases de una clase dada.

Véase también metaclass.

Semántica

Las subclases de una clase dada pueden considerarse a su vez instancias de una metaclass. Esta metaclass es lo que se denomina un supratipo. Por ejemplo, la clase **Árbol** puede tener las subclases **Roble**, **Olmo** y **Sauce**. Consideradas como objetos, estas subclases son instancias de la metaclass **EspecieDeÁrbol**. Entonces **EspecieDeÁrbol** es un supratipo de nivel superior a **Árbol**.

Notación

Un supratipo se representa mediante una clase con el estereotipo «**powertype**». Está conectado a un conjunto de rutas de generalización mediante una flecha discontinua que tiene el estereotipo «**powertype**» asociado a la flecha. (Figura 13.177).

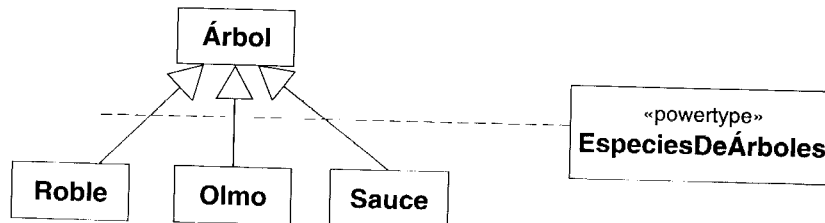


Figura 13.177 Supratipo

tiempo

Valor que representa un instante de tiempo absoluto o relativo.

Véase expresión temporal.

tiempo de análisis

Tiempo durante el cual se realiza la actividad de análisis en un proceso de desarrollo de software. No suponga que todo el análisis de un sistema se realiza al mismo tiempo o que precede a otras actividades como el diseño o la implementación, ya que las diversas actividades pueden ser secuenciales para un determinado elemento, pero cuando se trata del sistema completo las actividades se entremezclan.

Véase tiempo de diseño, tiempo de modelado.

tiempo de compilación

Se refiere a algo que ocurre durante la compilación de un módulo de software.

Véase tiempo de ejecución, tiempo de modelado.

tiempo de ejecución

Período de tiempo durante el cual se ejecuta un programa de computador. Contrastar con: tiempo de modelado.

tiempo de modelado

Denota todo aquello que se produce durante una actividad de modelado correspondiente al proceso de desarrollo del software. Se incluye el análisis y el diseño. Nota de utilización: cuando se discuten sistemas de objetos, suele ser importante distinguir entre los problemas del tiempo de modelado y del tiempo de ejecución.

Véase también proceso de desarrollo, fases de modelado.

tiempo de transición

Véase marca de tiempo.

tiempo de diseño

Se refiere a lo que ocurre durante la actividad de diseño del proceso de desarrollo de software. Contrastar con: tiempo de análisis.

Véase tiempo de modelado, fases de modelado.

tipo

Como adjetivo: El clasificador declarado que tienen que contener el valor de un atributo, parámetro o variable. El valor real tiene que ser una instancia del tipo o de uno de sus descendientes.

Como sustantivo: Estereotipo de una clase que se emplea para especificar un conjunto de instancias (tales como objetos) junto con las operaciones aplicables a esos objetos. Un tipo no puede contener métodos. Contrastar con: interfaz, clase de implementación.

Tipo y clase de implementación

Las clases se pueden estereotipar como tipos o clases de implementación (aunque también pueden quedar sin ser diferenciadas). Los tipos se emplean para especificar un dominio de objetos, junto con los parámetros aplicables a los objetos, sin definir la implementación física de los objetos o de sus operaciones. Los tipos no pueden contener métodos pero pueden proporcionar especificaciones de comportamiento para sus operaciones. También pueden incluir atributos y asociaciones para especificar el comportamiento de sus operaciones. Los atributos y asociaciones de un tipo no determinan la implementación de sus objetos.

Una clase de implementación define la estructura física de los datos y métodos de sus objetos, tal como se implementan en lenguajes tradicionales como C++ y SmallTalk. Se dice que la clase de implementación realiza un tipo si la clase de implementación incluye todas las operaciones del tipo con el mismo comportamiento. Una clase de implementación puede realizar múltiples tipos y múltiples clases de implementación pueden realizar un mismo tipo. Los atributos y asociaciones de una clase de implementación no tienen por qué ser los mismos que los de un tipo que realice. La clase de implementación puede proporcionar métodos para sus operaciones en términos de sus atributos y asociaciones físicas y puede declarar operaciones adicionales que no se encuentren en ningún tipo.

Un objeto sólo puede ser instancia de una clase de implementación, que especifica la implementación física del objeto. Sin embargo, un objeto puede ser instancia de múltiples tipos. Si el objeto es una instancia de una clase de implementación, entonces la clase de implementación tiene que realizar los tipos de los cuales sea instancia el objeto. Si se dispone de clasificación dinámica, entonces un objeto puede ganar y perder tipos a lo largo de su ciclo de vida. Los tipos que se emplean de este modo caracterizan el rol cambiante que puede adoptar un objeto para luego abandonarlo.

Aun cuando el uso de los tipos y de las clases de implementación es diferente, su estructura interna es la misma, así que se modelan como estereotipos de clases. Admiten la generalización en toda su extensión, así como el principio de capacidad de sustitución y la herencia de atributos, asociaciones y operaciones. Sin embargo, los tipos sólo pueden especializar a otros tipos y las clases de implementación sólo pueden especializar a otras clases de implementación. Los tipos sólo pueden estar relacionados con clases de implementación por realización.

Notación

Las clases sin diferenciar se muestran sin palabra reservada. Los tipos se muestran con la palabra clave «**type**» y las clases de implementación se muestran con la palabra clave «**clase de implementación**». La Figura 13.178 muestra un ejemplo.

La implementación de un tipo por parte de una clase de implementación se modela empleando la relación de realización, que se denota mediante una línea discontinua con una punta de flecha triangular rellena (una “flecha discontinua de generalización” o una “dependencia con

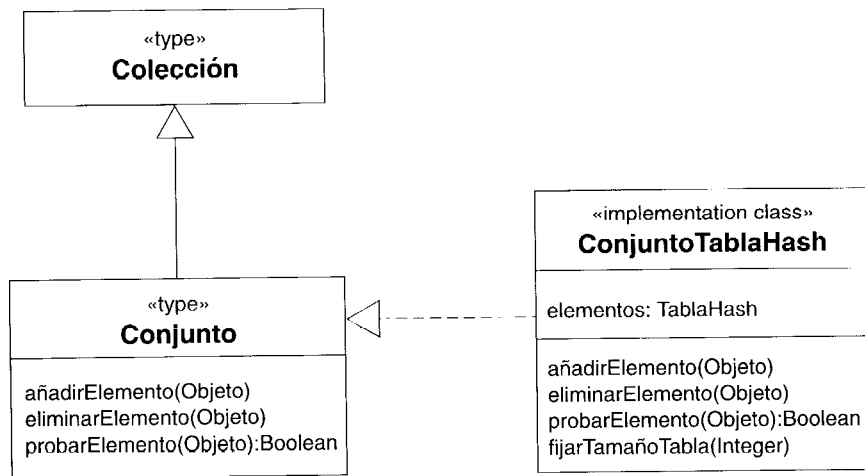


Figura 13.178 Notación para los tipos y clases de implementación

punta de flecha rellena”). Este símbolo implica la herencia de operaciones pero no de estructura (atributos o asociaciones). La realización puede emplearse entre dos clasificadores cualesquiera para los cuales uno admita las operaciones del otro, pero la forma habitual en que se usa es la implementación de un tipo por parte de una clase de implementación.

Discusión

Los tipos y las clases de implementación son conceptos relativamente limitados, destinados a los lenguajes de programación convencionales como C++ y SmallTalk. El concepto de clase de UML se puede emplear directamente de una forma más general. UML admite tanto la clasificación múltiple como la clasificación dinámica, lo cual elimina la necesidad de distinguir entre tipos y clases de implementación. Las diferencias, sin embargo, son útiles para diseñar código destinado a lenguajes convencionales.

tipo de dato

Un descriptor de un conjunto de valores que carecen de identidad (existencia independiente y la posibilidad de efectos secundarios). Los tipos de datos incluyen tipos primitivos predefinidos y tipos definibles por el usuario. Los tipos primitivos son números, cadenas, y tiempo.

Los definidos por el usuario son enumeraciones. Los tipos anónimos de datos previstos para la implementación en un lenguaje de programación se pueden definir usando tipos del lenguaje.

Véase también clasificador, identidad.

Semántica

Los tipos de datos son los primitivos predefinidos que se necesitan como base de los tipos definidos por el usuario. Su semántica se define matemáticamente fuera de los mecanismos de

construcción de tipos de un lenguaje. Los números están predefinidos. Incluyen números enteros y reales. También están predefinidas las cadenas. Estos tipos de datos no son definidos por el usuario.

Los tipos enumeración son conjuntos finitos, definidos por el usuario, de elementos nombrados que tienen un orden definido entre sí mismos y ninguna otra propiedad computacional. Un tipo enumeración tiene un nombre y una lista de las constantes de la enumeración. El tipo **booleano** se define con los literales de la enumeración **false** y **true**.

Las operaciones se pueden definir en los tipos de datos, y las operaciones pueden tener tipos de datos como parámetros. Debido a que un tipo de datos no tiene ninguna identidad y es sólo un valor, las operaciones en los tipos de datos no los modifican; en su lugar, simplemente vuelven valores. No tiene ningún sentido hablar de crear un nuevo valor del tipo de datos, porque carece de identidad. Todos los valores del tipo de datos se encuentran (conceptualmente) predefinidos. Una operación en un tipo de datos es una consulta que puede no cambiar el estado del sistema pero puede retornar un valor.

Un tipo de dato se puede también describir por un tipo del lenguaje —una expresión de un tipo de datos en un lenguaje de programación—. Dicha expresión señala un tipo de dato anónimo en el lenguaje en cuestión. Por ejemplo, la expresión **Persona* (*) (String)** denota una expresión de tipo en C++ que no se corresponde a ningún tipo simple de datos con un nombre.

tipo del lenguaje

Dícese de un tipo de datos anónimo que se define en la sintaxis de un lenguaje de programación.

Véase también tipo de dato.

Semántica

Un tipo del lenguaje es una expresión que debe ser interpretada como un tipo de dato de algún lenguaje de programación. Se puede emplear como tipo de un atributo, variable o parámetro. No posee nombre y no declara un nuevo tipo de dato.

Por ejemplo, el tipo de dato de C++ "**Persona* (*) (Contrato*,int)**" se podría definir como un tipo del lenguaje C++.

La intención de un tipo del lenguaje es la implementación en un determinado lenguaje de programación. Para relaciones más lógicas deben emplearse asociaciones.

tipo primitivo

Este término denota un tipo de dato básico predefinido, tal como un entero o una cadena.

Véase también enumeración.

Semántica

Las instancias de los tipos primitivos carecen de identidad. Si dos instancias tienen la misma representación, entonces son indistinguibles y se pueden pasar por valor sin pérdida de información.

Entre los tipos primitivos se cuentan los números y las cadenas, así como posiblemente otros tipos de datos dependientes del sistema tales como fechas y valores monetarios, cuya semántica está predefinida fuera de UML.

Se supone que los tipos primitivos tendrán una fuerte correspondencia con los que se hallan en el lenguaje de programación que se tenga como destino.

Véase también enumeraciones, que son tipos de datos definibles por el usuario y que no son tipos primitivos predefinidos.

transición

Relación entre dos estados dentro de una máquina de estados que indica que un objeto del primer estado va a ejecutar las acciones especificadas para después pasar al segundo estado cuando se produzca un evento especificado y se cumplan las condiciones de guarda. En este cambio de estado se dice que se dispara la transición. Las transiciones simples poseen un único estado de origen y un único estado blanco. Una transición compleja posee más de un origen y/o más de un estado destino. Representa un cambio en el número de estados activos concurrentemente o una bifurcación o reunificación de control. Las transiciones internas poseen estado de origen pero no estado destino. Representan la respuesta a un evento sin cambio de estado. Los estados y las transiciones son los vértices y nodos de las máquinas de estados.

Véase también máquina de estados.

Semántica

Las transiciones representan rutas potenciales entre estados dentro de la historia de los objetos, así como las acciones que se llevan a cabo al cambiar de estado. Una transición indica la forma en que responde a un evento un objeto que está en un cierto estado. Los estados y las transiciones son los vértices y arcos de una máquina de estados que describe las posibles historias de las instancias de un clasificador.

Estructura

Toda transición puede poseer un estado de origen, un evento disparador, una condición de guarda, una acción y un estado destino. Las transiciones pueden carecer de alguno de estos elementos.

Estado de origen. El estado de origen es el estado que se ve afectado por la transición. Si un objeto se encuentra en el estado de origen, se puede disparar una transición saliente si el objeto recibe el evento disparador de la transición y se cumple la condición de guarda (si existe).

Estado destino. El estado destino es el estado que está activo una vez finalizada la transición. Es el estado al que pasa el objeto maestro. El estado destino no se usa en las transiciones internas, que no llevan a cabo cambios de estado.

Evento disparador. El evento disparador es el evento cuya recepción por parte del objeto cuando se encuentra en el estado de origen hace que la transición pueda dispararse, siempre y cuando se cumpla la condición de guarda. Si el evento posee parámetros, sus valores estarán disponibles para la transición y se pueden utilizar en expresiones para la condición de guarda y en acciones. El evento que dispara una transición pasa a ser el evento actual y es posible acceder a él desde las acciones subsiguientes que forman parte de la ejecución hasta finalizar iniciada por ese evento.

Una transición que carece de un evento disparador explícito es lo que se denomina una transición de finalización (o transición sin disparador) y se dispara implícitamente al finalizar cualquier actividad interna del estado. Si un estado no posee actividad interna o estados anidados, entonces cuando se entra en el estado después de haber ejecutado cualquier acción de entrada se dispara inmediatamente una transición de finalización. Observe que una transición de finalización tiene que satisfacer su condición de guarda para poder dispararse. Si la condición de guarda es falsa al producirse la terminación, entonces el evento implícito de finalización se consume y la transición no se disparará más tarde aun cuando la condición de guarda pase a ser verdadera. (Este tipo de comportamiento se puede modelar, de otro modo, empleando un evento de cambio.)

Observe que todas las apariciones de un evento dentro de una máquina de estados tienen que tener la misma signatura.

Condición de guarda. La condición de guarda es una expresión booleana que se evalúa cuando se dispara una transición por haber manejado un evento, incluyendo los eventos implícitos de finalización que se producen en una transición de finalización. Si la máquina de estados está realizando un paso que se ejecuta hasta finalizar cuando se produce un evento, entonces se guarda el evento hasta que el paso ha finalizado y la máquina de estados está en reposo. En caso, contrario, se maneja el suceso inmediatamente. Si la expresión produce el valor verdadero al ser evaluada, la transición puede dispararse. Si la expresión toma el valor falso, entonces la transición no se dispara. Si no hay ninguna expresión que pueda dispararse, se ignora el evento. Esto no es un error. Es posible que múltiples transiciones que posean distintas condiciones de guarda sean disparadas por el mismo evento. Si se produce el evento, entonces se comprueban todas las condiciones de guarda. Si hay más de una condición de guarda que sea verdadera, sólo se dispara una transición. Si no se da una regla de prioridad, la elección de la transición que se dispara no es determinista.

Observe que la condición de guarda sólo se evalúa una vez, en el momento en que se maneja el evento. Si la condición produce el valor falso al ser evaluada y después se vuelve verdadera, la transición no se disparará mientras no se produzca otro evento y la condición sea verdadera en ese momento. Observe que una condición de guarda no es la forma correcta de monitorizar continuamente un valor. Para una situación como ésta debería emplearse un evento de cambio.

Si una transición no posee una condición de guarda, entonces la condición de guarda se trata como si fuera verdadera y la transición se activa si se produce un evento disparador. Si están habilitadas varias transiciones, sólo se dispara una de ellas. La selección puede no ser determinista.

Por comodidad, se puede descomponer una condición de guarda en una serie de condiciones de guarda más sencillas. De hecho, se pueden ramificar varias condiciones de guarda a partir de un único evento disparador o condición de guarda. Toda ruta que pase por el árbol representa una única transición disparada por el (único) evento disparador con una condición de guarda efectiva diferente, que es la conjunción (“and”) de las condiciones de guarda existentes a lo largo de su trayectoria. Todas las expresiones que haya a lo largo de la trayectoria se evaluarán antes de seleccionar la transición que se dispare. Una transición no puede dispararse parcialmente. De hecho, un conjunto de transiciones independientes puede compartir una parte de su descripción. La Figura 13.179 muestra un ejemplo.

Observe que los árboles de condiciones de guarda y la capacidad para ordenar transiciones que se pueden disparar son únicamente una cuestión de comodidad, porque se podría conseguir el mismo efecto mediante un conjunto de transiciones independientes, dotando a cada una de ellas de su propia condición de guarda disjunta.

Acción. Una transición puede contener una expresión que describe una acción. Se trata de una expresión para una computación procedimental que puede afectar al objeto que posee la máquina de estados (e indirectamente a otros objetos a los que tenga acceso). Esta expresión puede hacer uso de los parámetros del evento disparador, así como de los atributos y asociación del objeto que la posee. El evento disparador está disponible como evento actual durante todo el paso ejecutado hasta finalizar que ha sido iniciado por dicho evento, incluyendo los segmentos sin disparador y las acciones de entrada y de salida posteriores.

Una acción puede ser una secuencia de acción. Las acciones son atómicas, esto es, no se puede imponer su terminación externamente y tienen que ser ejecutadas por completo antes de que sea posible manejar ninguna otra acción o evento. Las acciones deberían tener una duración mínima, para evitar bloquear la máquina de estados. Los eventos que se reciben durante la ejecución de una acción se guardan hasta que finaliza la acción, momento en que son evaluados.

Ramificaciones. Por comodidad, varias transiciones que compartan el mismo evento disparador pero tengan diferentes condiciones de guarda se pueden agrupar en el modelo y en la notación para evitar la duplicación del disparador o de la parte común de la condición de guarda.

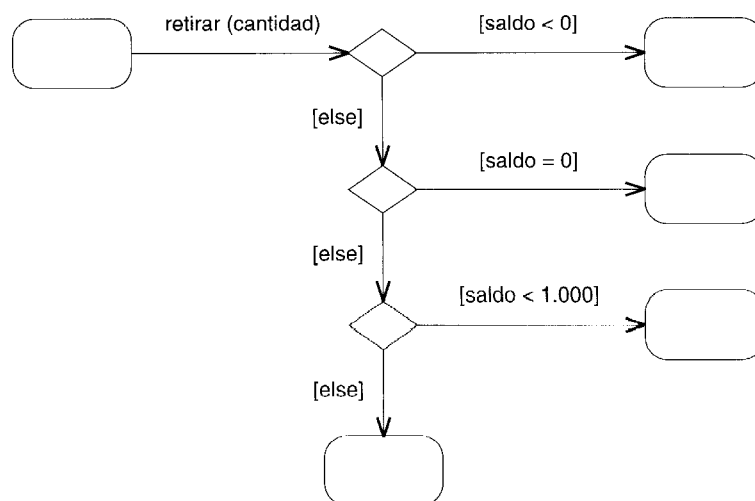


Figura 13.179 Árbol de condiciones de guarda

Esto no pasa de ser una forma cómoda de representación y no afecta a la semántica de las transiciones.

Véase bifurcación para más detalles de la representación y de la notación.

Notación

Las transiciones se representan mediante una flecha continua que va desde un estado (el de *origen*) a otro estado (el estado *destino*), etiquetadas con una cadena de transición. La Figura 13.180 muestra una transición entre dos estados y una que se ha fragmentado en segmentos.

Las transiciones internas se representan mediante una cadena de transición situada dentro de un símbolo de estado. Las cadenas de transición tienen el formato siguiente:

nombre :_{opt} nombre-evento_{opt} (lista-parámetros)_{opt} [condición-guarda]_{opt}
/ lista-acciones_{opt}

El *nombre* se puede utilizar para hacer referencia a la transición en las expresiones, especialmente para formar marcas de tiempo. Va seguido por dos puntos.

El *nombre-evento* da nombre a un evento y va seguido por sus parámetros. La lista de parámetros se puede omitir si no hay parámetros. El nombre del evento y la lista de parámetros se omiten en las transiciones de finalización. La lista-parámetros posee el formato siguiente:

nombre :tipo_{lista}

La *condición-guarda* es una expresión booleana escrita en términos de los parámetros del evento disparador y de los atributos y enlaces del objeto que describe la máquina de estados. La condición de guarda también puede implicar comprobaciones del estado de estados concurrentes de la máquina actual o estados indicados especialmente de algún objeto que se pueda alcanzar; entre los ejemplos se cuentan [**in** estado1] y [**not in** estado2]. Los nombres de estados pueden estar completamente calificados con los nombres de los estados anidados que los contienen, lo cual da lugar a nombres de ruta de la forma Estado1::Estado2::Estado3. Esto se puede utilizar si aparece el mismo nombre de estado en distintas regiones de estados compuestos dentro de la máquina global.



Figura 13.180 Transiciones

La *lista-acciones* es una expresión procedimental que se ejecuta siempre y cuando llegue a dispararse la transición. Puede estar escrita en términos de operaciones, atributos y enlaces del objeto poseedor y también empleando los parámetros del evento disparador. Entre las acciones se pueden incluir llamadas, envíos y otras clases de acciones. La lista-acciones puede contener más de una cláusula de acción separada por delimitadores formados por el signo de punto y coma:

acción_{lista}.

Ramificaciones. Una transición puede incluir un segmento con un evento disparador seguido por un árbol de estados de unión, que se dibujan como circulitos. Esto es equivalente a un conjunto de transiciones individuales, una para cada posible ruta a través del árbol, cuya condición de guarda es el “and” de todas las condiciones que se vayan encontrando a lo largo de la ruta. Sólo puede tener una acción el segmento final de la ruta.

Alternativamente, se puede dibujar un estado de unión como un rombo, si representa una ramificación o una reunificación. No hay diferencia de significados.

Discusión

Las transiciones representan un cambio atómico de un estado a otro, acompañado posiblemente por una acción atómica. Las transiciones no se pueden interrumpir. Las acciones de la transición deberían ser computaciones cortas, normalmente triviales, tales como sentencias de asignación y sencillos cálculos aritméticos.

transición abreviada

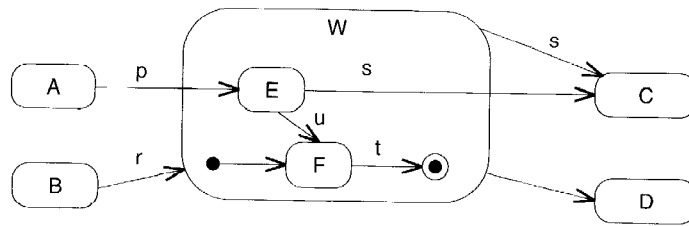
Notación que indica que una transición pasa directamente a un estado compuesto, pero se suprimen los detalles.

Véase también estado abreviado externo.

Notación

Una *abreviatura de estado* se representa mediante una pequeña línea vertical que se dibuja dentro de los límites del estado que la contiene (Figura 13.181). La abreviatura puede estar etiquetada con el nombre del estado, pero es frecuente que se omita cuando se suprimen los detalles. Indica que una transición está conectada a un estado interno suprimido. Las abreviaturas no se utilizan para transiciones a estados iniciales ni para salir de estados finales. Una abreviatura muestra que hay subestados adicionales en el modelo, pero faltan en el diagrama.

Un estado abreviado externo dentro de un estado de referencia a submáquina (Figura 13.91) hace referencia a un estado perteneciente a la definición de submáquina correspondiente (Figura 13.92). No se trata de un caso en que se hayan suprimido los detalles y es imprescindible incluir el nombre de la abreviatura.



se puede abstraer de este modo

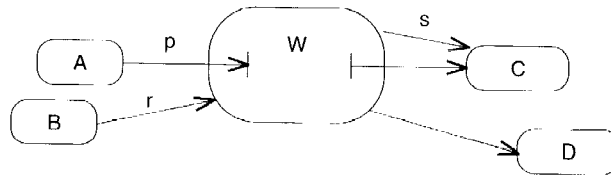


Figura 13.181 Transición abreviada

transición compleja

Transición con más de un estado origen y/o más de un estado destino. Representa una respuesta a un evento que causa un cambio en la cantidad de concurrencia. Es una sincronización de control, una división del control, o ambas, dependiendo del número de orígenes y destinos.

Véase también bifurcación, división, estado compuesto, reunificación, unión.

Semántica

A un alto nivel, un sistema pasa por una serie de estados, pero la visión monolítica de un sistema que tiene un único estado es demasiado restrictiva para grandes sistemas en los que interviene la distribución y la concurrencia. Un sistema puede estar en muchos estados simultáneamente. El conjunto de estados activos se denomina configuración del estado activo. Si está activo un estado anidado, el estado que lo contiene también está activo. Si el objeto permite concurrencia, entonces puede haber más de un subestado concurrente activo.

En muchos casos la actividad de un sistema puede modelarse como un conjunto de hilos de control que evolucionan independientemente unos de otros o que interactúan de forma limitada. Cada transición afecta, al menos, a unos pocos estados de la configuración del estado activo. Cuando se dispara una transición, los estados no afectados por la misma permanecen activos. El progreso de los hilos en un momento determinado se puede capturar como un subconjunto de estados dentro de la configuración del estado activo, de forma que exista un subconjunto por cada hilo. Cada conjunto de estados evoluciona de forma independiente en respuesta a eventos. Si el número de estados activos es constante, el modelo de estados no es sino una colección fija de máquinas de estados que interactúan. Sin embargo, en general el número de estados (y por tanto el número de hilos de control) puede variar a lo largo del tiempo. Se puede pasar de un estado a dos o más estados concurrentes (división del control), y se puede pasar de dos o más estados concurrentes a un único estado (unión de control). La máquina de estados del sistema controla el número de estados concurrentes y su evolución.

Una transición compleja es una transición hacia o desde un conjunto de subestados. Una transición compleja tiene más de un estado origen y/o más de un estado destino. Si tiene múltiples estados origen representa una unión de control, mientras que si tiene múltiples estados destino representa una división del control; si cuenta con múltiples estados origen y destino representa una sincronización de hilos paralelos.

Si una transición compleja tiene múltiples estados origen, todos ellos deben estar activos antes de que la transición sea candidata a ser disparada, siendo irrelevante el orden en que llegan a estar activos. Si todos los estados origen están activos y se produce el evento, se dispara la transición y puede ejecutarse si su condición de guarda es verdadera. Cada transición se dispara a partir de un único evento, incluso aunque haya múltiples estados origen, pues en UML no hay ocurrencia simultánea de eventos: cada evento debe disparar una transición distinta, aunque a los estados resultantes puede suceder una unión de estados.

Si una transición compleja con múltiples estados carece de un evento que la dispare (es decir, si es una transición de finalización) entonces se dispara cuando todos sus estados origen explícitos estén activos. Se lleva a cabo si en ese momento su condición de guarda es verdadera.

Cuando se lleva a cabo una transición compleja, todos los estados origen y todos sus iguales dentro del mismo estado compuesto dejan de estar activos y pasan a estarlo todos los estados destino y sus iguales.

En situaciones más complicadas, puede expandirse la condición de guarda para permitir la ejecución de la transición cuando esté activo algún subconjunto de los estados.

Ejemplo

La Figura 13.182 muestra un típico estado compuesto concurrente con transiciones complejas de entrada y de salida. La Figura 13.183 muestra una típica historia de ejecución de esta máquina (los estados activos se representan mediante letra en azul). La historia muestra la variación en el número de estados activos a lo largo del tiempo.

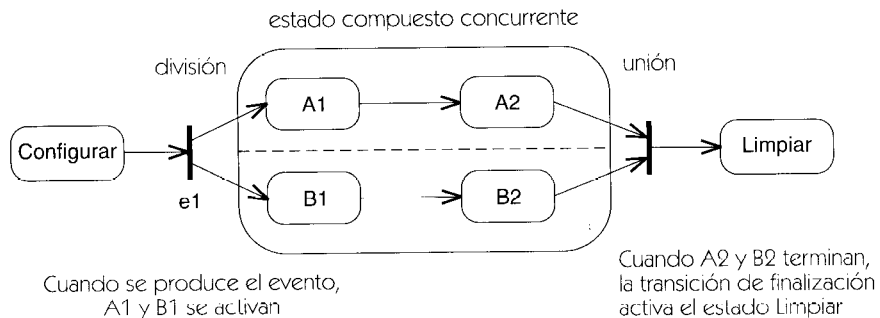


Figura 13.182 División y unión

Estados concurrentes

A menos que una máquina de estados haya sido cuidadosamente estructurada, un conjunto de transiciones complejas puede derivar en inconsistencias, incluyendo interbloques, ocupa-

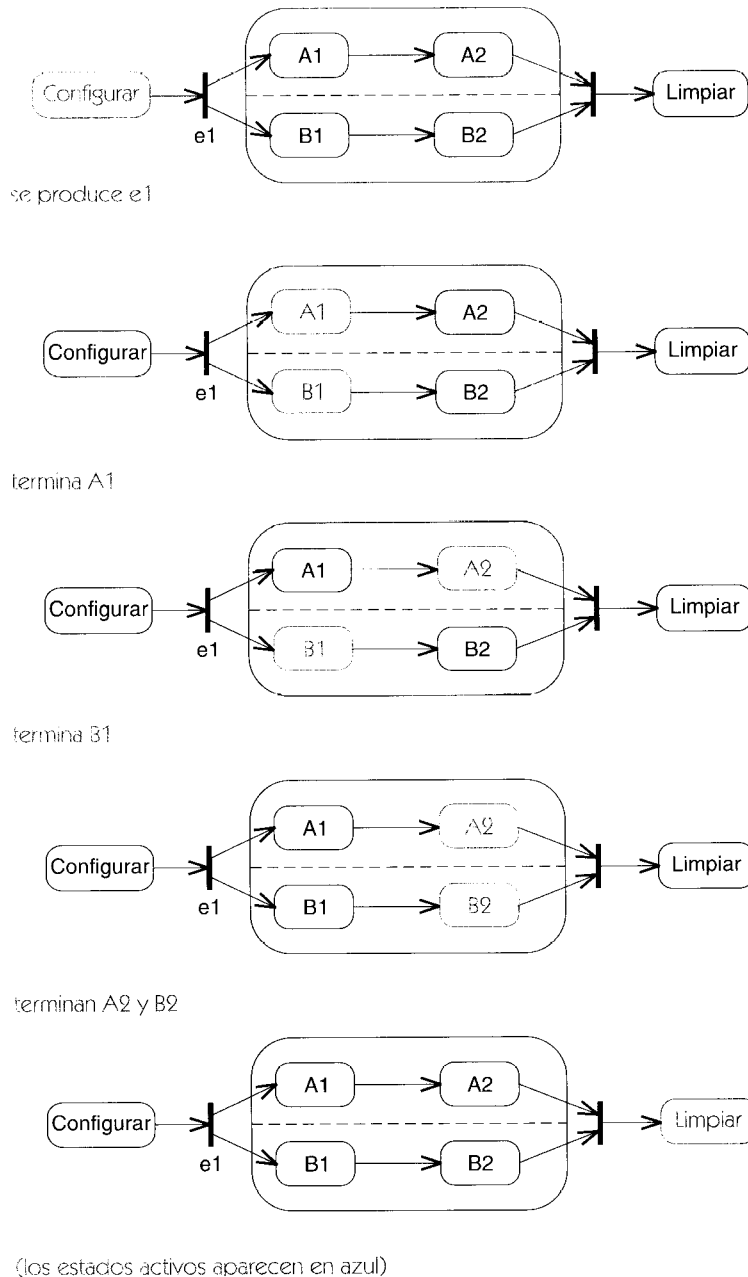


Figura 13.183 Historia de estados activos en una máquina de estados concurrente

ción múltiple de un estado, y otros problemas. El problema ha sido estudiado en profundidad a la luz de la teoría de redes de Petri, y la solución más habitual es imponer reglas de formación correcta a la máquina de estados para evitar el riesgo de inconsistencias, que son algo así como las reglas de “programación estructurada” para las máquinas de estados. Hay numerosos modelos, cada uno con sus ventajas e inconvenientes. Las reglas adoptadas por UML requieren que la máquina de estados se descomponga en estados más pequeños utilizando una especie de

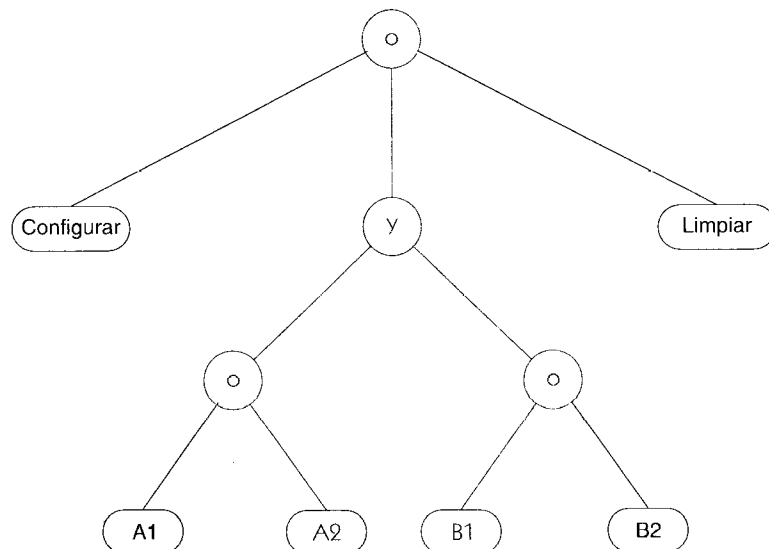
árbol Y/O. La ventaja es que una estructura bien anidada resulta sencilla de establecer, de mantener y de comprender. La desventaja es que hay ciertas configuraciones que están prohibidas aunque tienen significado. Haciendo balance, podemos decir que es algo similar a las soluciones de compromiso tomadas en referencia a las cláusulas “goto” en la programación estructurada.

Un estado complejo puede descomponerse en un conjunto de subestados mutuamente exclusivos (descomposición “O”) o en un conjunto de subestados concurrentes (descomposición “Y”). La estructura es recursiva. Generalmente, las capas “Y” se alternan con las capas “O”. Una capa “Y” representa una descomposición concurrente —todos los subestados están activos concurrentemente—, mientras que una capa “O” representa una descomposición secuencial —sólo hay un estado activo al mismo tiempo—. Se puede obtener un conjunto legítimo de subestados concurrentes expandiendo recursivamente los nodos del árbol empezando por la raíz, remplazando todos los estados “O” por todos sus hijos y remplazando todos los estados “Y” por todos sus hijos. El resultado obtenido se corresponde con la estructura anidada de los diagramas de estados.

Ejemplo

La Figura 13.184 muestra un árbol Y/O de estados correspondiente a la máquina de estados de la Figura 13.182. Se muestra un típico conjunto de estados activos en azul que se corresponde con el tercer paso de la Figura 13.183.

Si una transición entra en una región concurrente, entra en todos sus subestados, mientras que si entra en una región secuencial, entra exactamente en un subestado. El estado activo de una región secuencial puede cambiar, pero en una región concurrente todos los subestados concurrentes permanecen activos durante el tiempo en que la región está activa, aunque cada subestado concurrente se descompone a su vez en una región secuencial.



(El conjunto típico de estados activos aparece en azul)

Figura 13.184 Árbol Y/O de estados anidados

Por consiguiente, una transición simple (una que tiene sólo una entrada y una salida) debe conectar dos estados de la misma región secuencial o separados sólo por niveles "O". Una transición compleja debe conectar todos los subestados de una región concurrente con un estado externo a la región concurrente (se omiten deliberadamente casos más complicados, pero seguirían los mismos principios expuestos más arriba). En otras palabras, una transición de entrada en una región concurrente debe entrar en cada subestado, y una transición de salida de una región concurrente debe salir de todos los subestados.

Existe una representación abreviada: si una transición compleja entra en una región concurrente pero omite una o más de sus subregiones, existe una transición implícita al estado inicial de cada subregión omitida. Si alguna subregión no tiene estado inicial, el modelo está mal formado. Si una transición compleja sale de una región concurrente, existe una transición implícita para cada subregión omitida. Si se dispara la transición, finaliza toda actividad dentro de la subregión —es decir, representa una salida forzada—. Una transición puede conectarse con la propia región concurrente, lo que implica una transición al estado inicial de cada subregión —una situación muy común de modelado—. De forma parecida, una transición desde una región concurrente que engloba diversas subregiones provoca una salida forzada de cada una de las subregiones (si tiene un evento disparador) o una espera hasta que finaliza cada subregión (si no existen disparadores).

Las reglas sobre transiciones complejas aseguran que no pueda haber combinaciones de estados carentes de significado activas simultáneamente. Un conjunto de subestados concurrentes es una partición del estado compuesto que las engloba, donde o bien están todos activos o bien no lo está ninguno.

Hilo condicional

En un grafo de actividades, un segmento que sale de una división puede tener una condición de guarda. Esto es un hilo condicional. Cuando se dispara la transición, el hilo que comienza por el segmento que contiene la condición de guarda se inicia sólo si se satisface la condición de guarda. Los segmentos sin guardas siempre se inician cuando se dispara la transición. La concurrencia en un grafo de actividades debe estar bien anidada: cada división debe corresponderse con una unión posterior. Cuando un hilo condicional no puede iniciarse debido a que su condición de guarda se evalúa a falso, la actividad del segmento de entrada correspondiente en la unión posterior se considera completa, esto es, la transición no espera un flujo de control de dicho hilo condicional. Si ningún hilo de una división puede iniciarse, el control pasa inmediatamente a la unión.

Un hilo condicional es equivalente a un grafo con una bifurcación y una reunificación rodeando la parte condicional del grafo de actividades.

Ejemplo

La Figura 13.185 muestra un grafo de actividades con dos hilos condicionales, que representa el procedimiento de admisión que sigue una línea aérea. Antes de que nada pueda ocurrir, el cliente debe presentar el billete. A partir de ese momento hay tres hilos concurrentes, dos de los cuales son condicionales, ya que la asignación de asiento se realiza siempre. La comprobación del equipaje sólo se realiza si el pasajero tiene equipaje, y el pasaporte sólo se comprueba si el vuelo es internacional. Cuando se completan todos los hilos el control se une en un único hilo

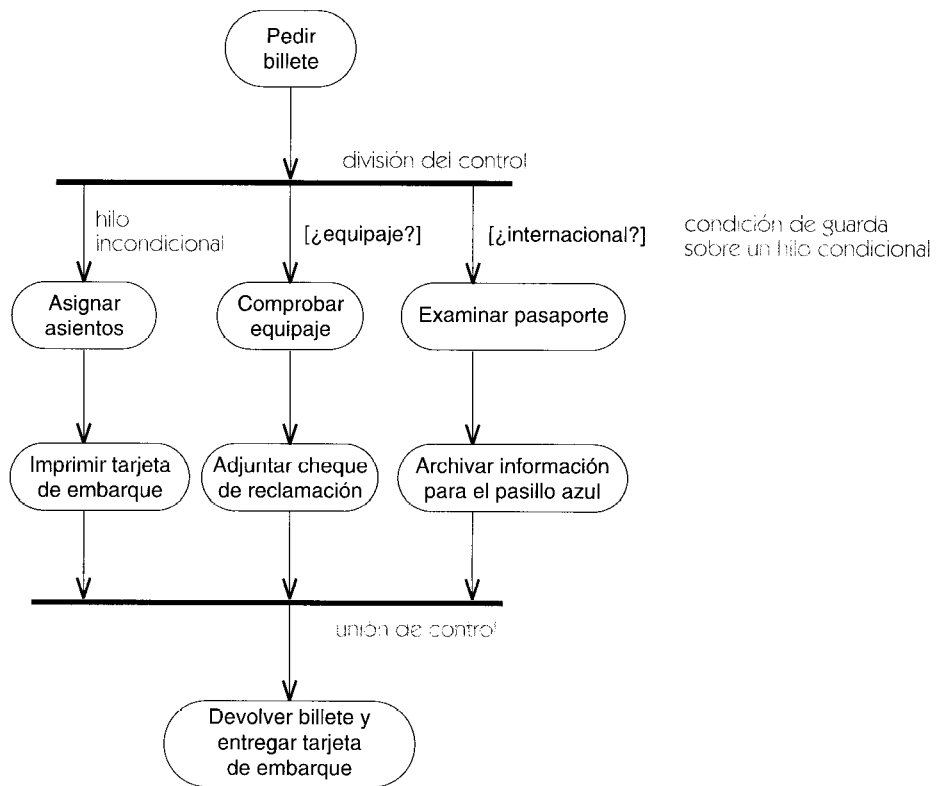


Figura 13.185 Hilos condicionales

en el cual se devuelve el billete al pasajero y se le entrega la tarjeta de embarque. Si no se inicia algún hilo condicional, se considera completado en la unión posterior.

Notación

Una transición compleja se representa mediante una pequeña barra gruesa (una barra de sincronización, que puede representar sincronización, división o ambas cosas). La barra puede tener una o más flechas de transición (dibujadas con línea continua) que van de los estados origen a la barra, y una o más flechas de transición desde la barra a los estados destino. Se puede mostrar una etiqueta de transición cerca de la barra para describir el evento disparador, la condición de guarda o la descripción de las acciones que se producen bajo la transición. Las flechas individuales no tienen sus propias cadenas de transición, ya que son meras partes de la transición global.

Ejemplo

La Figura 13.186 muestra la máquina de estados de la Figura 13.182 a la que se ha añadido una transición de salida adicional. También muestra una división implícita desde el estado **Configurar** hacia los estados iniciales de cada subregión y una unión implícita desde los estados finales de cada subregión hacia el estado **Limpiar**.

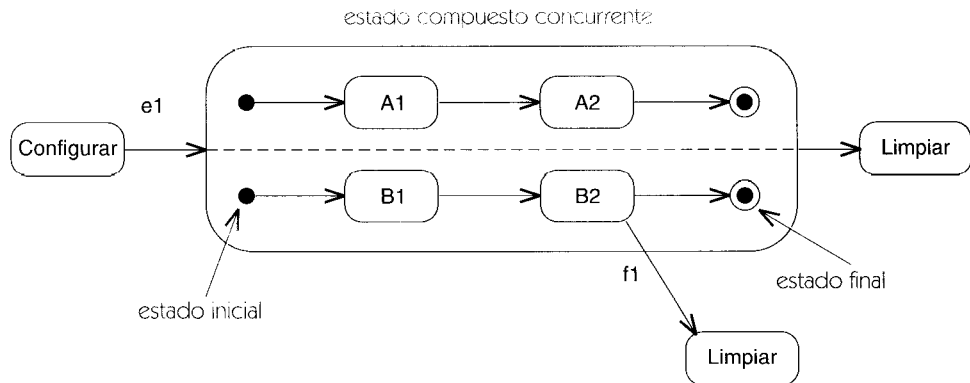


Figura 13.186 Transiciones complejas (división, unión)

Si se produce el evento **f1** cuando está activo el estado **B2**, se produce una transición al estado **Limpiar**. Esta transición es una unión implícita, ya que produce la finalización del estado **A2** a la vez que finaliza el estado **B2**.

transición de finalización

Transición que carece de un evento explícito de disparo y que se dispara por la finalización de la actividad del estado origen.

Véase también actividad, disparador, transición.

Semántica

Una transición de finalización se representa mediante una transición que no tiene un evento explícito de disparo. La transición se dispara implícitamente cuando su estado origen ha completado toda su actividad (incluyendo sus estados anidados). En un estado compuesto, la finalización de la actividad supone alcanzar el estado final. Si un estado no tiene actividad interna ni estados anidados, la transición de finalización se dispara inmediatamente después de la ejecución de la acción de entrada y de la acción de salida sin la intervención de otros eventos.

Si un estado tiene estados anidados o actividad interna pero carece de transiciones de salida disparadas por eventos, no está garantizada la ejecución de la transición de salida. Un evento podría disparar una transición sobre uno de los estados anidados mientras el estado que lo engloba está a la espera de completar su actividad, en cuyo caso la transición de finalización no se llevaría a cabo.

Una transición de finalización puede tener una condición de guarda y una acción. Normalmente no es deseable la existencia de una única transición de finalización con condición de guarda ya que si dicha condición es falsa nunca se dispara la transición (debido a que el disparador implícito sucede sólo una vez). Algunas veces esto puede ser útil para representar algún tipo de fallo, siempre que alguna transición saque al objeto de esa situación. Lo más común es que el conjunto de transiciones de finalización con condiciones de guarda tengan condiciones que cubran todas las posibilidades de forma que siempre se ejecute alguna de ellas cuando finalice el estado.

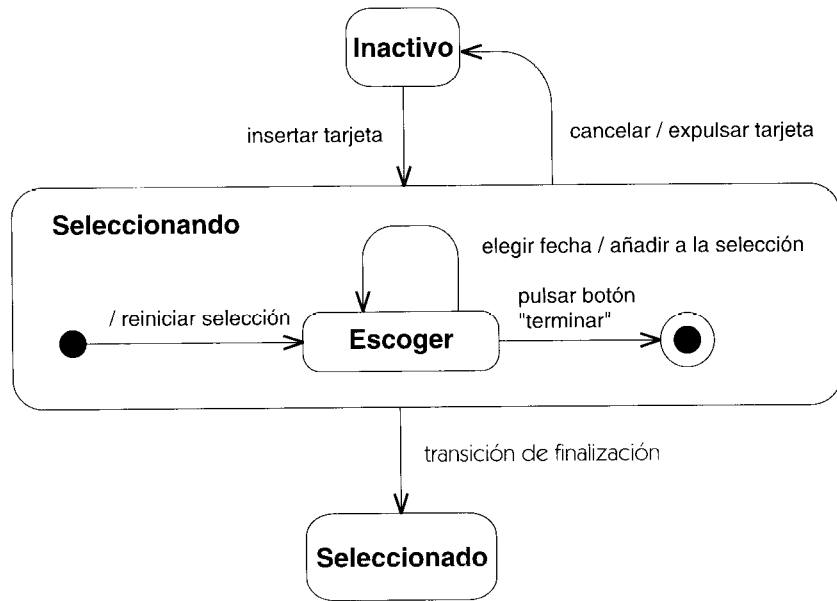


Figura 13.187 Transición de finalización

Las transiciones de finalización también se emplean para conectar estados iniciales y estados históricos con sus estados sucesores, ya que estos pseudoestados no pueden seguir activos después de la finalización de la actividad.

Ejemplo

La Figura 13.187 muestra un fragmento de una máquina de estados para una aplicación de un dispensador de entradas. El estado **Seleccionando** permanece activo durante todo el tiempo en que el cliente se encuentra eligiendo fechas. Cuando el cliente pulsa el botón “aceptar”, el estado **Seleccionando** alcanza su estado final, lo que dispara su transición de finalización y lleva a la máquina de estados al estado **Seleccionado**.

transición (fase)

Cuarta fase del proceso de desarrollo del software, durante la cual el sistema implementado se configura para su ejecución en un contexto del mundo real. Durante esta fase, se completa la vista de despliegue, junto con cualquiera de las vistas restantes que pudieran no haber finalizado en las fases anteriores.

Véase proceso de desarrollo.

transición interna

Se trata de una transición asociada a un estado que posee una acción pero no implica un cambio de estado.

Véase también máquina de estados.

Semántica

La transición interna permite que un evento dé lugar a una acción sin que haya un cambio de estado. Una transición interna posee un estado de origen pero no un estado destino. Si se dispara, se ejecuta la acción pero no cambia el estado aun cuando la transición interna esté conectada a un estado que englobe al estado actual y herede (la transición) de él. Por tanto, no se ejecuta ninguna acción de entrada ni de salida. En este aspecto, difiere de una autotransición, que da lugar a la salida de los estados anidados y a la ejecución de las acciones de salida y de entrada.

Notación

La transición interna se representa mediante una entrada de texto en el compartimento del estado destinado a la transición interna. La entrada posee la misma sintaxis que el rótulo de texto correspondiente a una transición externa. Como no hay un estado destino, no hay necesidad de asociarla a una flecha.

nombre de evento / expresión de acción

Los nombres de evento **entry**, **exit**, **do** e **include** son palabras reservadas y no se pueden emplear como nombres de evento. Estas palabras reservadas se emplean para declarar una acción de entrada, una acción de salida, la ejecución de una actividad o la ejecución de una submáquina, respectivamente. Por uniformidad, estas acciones especiales hacen uso de la sintaxis de transición interna para especificar la acción. Sin embargo, no son transiciones internas y las palabras reservadas no son nombres de eventos.

La Figura 13.188 muestra la notación.

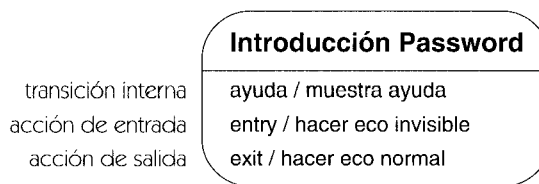


Figura 13.188 Sintaxis de transición interna

Discusión

Se puede pensar que una transición interna es una “interrupción” que da lugar a una acción sin afectar al estado actual; por tanto no invoca las acciones de salida ni las de entrada. La asociación de una transición interna a un estado compuesto es una buena forma de modelar una acción que debe suceder para un cierto número de estados pero no debe cambiar el estado activo —por ejemplo, para visualizar un mensaje de ayuda o para contar el número de apariciones de un cierto evento—. No es la forma correcta de modelar una salida por aborto o una excepción. Éstas deberían modelarse mediante transiciones a un nuevo estado, por cuanto su aparición invalida el estado actual.

transición simple

Transición con un único estado origen y un único estado destino. Representa la respuesta a un evento con un cambio de estado dentro de una región de estados mutuamente excluyentes. La cantidad de concurrencia no cambia cuando se ejecuta.

transición sin disparador

Transición que carece de un evento disparador explícito. Cuando sale de un estado normal, representa una transición de finalización, esto es, una transición que es disparada por la finalización de una actividad más que por un evento explícito. Cuando sale de un pseudoestado, representa un segmento de transición que se recorre automáticamente cuando el segmento precedente ha concluido su acción. Las transiciones sin disparador se utilizan para conectar estados iniciales y estados de historia con sus estados blanco.

traza

Dependencia que indica un proceso de desarrollo histórico o alguna otra relación más allá del modelo entre dos elementos que representan el mismo concepto sin reglas específicas para derivar uno a partir del otro. Se trata de la clase de dependencia menos específica y posee una semántica mínima. Su mayor aplicación es como recordatorio para el pensamiento humano durante el desarrollo.

Véase dependencia, modelo.

Semántica

Una traza es una variedad de dependencia que indica una conexión entre dos elementos que representan el mismo concepto en dos niveles diferentes de significado. No representa una semántica dentro de un modelo. Más bien, representa conexiones entre elementos de semántica diferente —esto es, de elementos pertenecientes a diferentes modelos situados en distintos planos de significado—. No hay una correspondencia explícita entre los elementos. Con cierta frecuencia, representa una conexión entre dos formas de capturar un concepto en distintas fases de desarrollo. Por ejemplo, dos elementos que sean variaciones de un mismo tema podrían estar relacionados por una traza. Una traza no representa una relación entre instancias en tiempo de ejecución. Más bien, es una dependencia entre elementos del modelo en sí.

Una aplicación importante de la traza es el seguimiento de requisitos que hayan ido cambiando a lo largo del desarrollo de un sistema. Las dependencias de traza pueden relacionar elementos de dos clases diferentes de modelos (tales como un modelo de caso de uso y un modelo de diseño) o pertenecientes a dos versiones de la misma clase de modelo.

Notación

Las trazas se denotan mediante una flecha de dependencia (una flecha discontinua el origen en el elemento más moderno y la punta de flecha en el elemento más antiguo) que posee la pala-

bra clave «**trace**». Normalmente, sin embargo, los elementos están en diferentes modelos que no se visualizan simultáneamente, así que en la práctica la relación se implementaría en forma de un hipervínculo dentro de alguna herramienta.

tupla

Lista ordenada de valores. Generalmente, el término implica que existe un conjunto de listas de forma similar. (Es un término matemático estándar.)

unidad de distribución

Un conjunto de objetos o componentes que se asignan a un proceso del sistema operativo o a un procesador como grupo. Una unidad de la distribución se puede representar por un compuesto de ejecución o por un agregado. Esto es un concepto de diseño en la vista de despliegue.

unión

Denota un cierto lugar de una máquina de estados, diagrama de actividades, o diagrama de secuencia en el cual se combinan dos o más hilos o estados concurrentes para formar un único hilo o estado; es una reunión o “confluencia”. Antónimo: bifurcación.

Véase transición compleja, estado compuesto.

Semántica

Diremos que una unión es una transición con dos o más estados de origen y un único estado blanco. Si todos los estados de origen están activados y se produce el evento disparador, se ejecuta la acción de transición y se activa el estado blanco. Los estados de origen tienen que estar en regiones diferentes de un estado compuesto concurrente.

Notación

Las uniones se representan mediante una barra gruesa con dos o más flechas entrantes de transición y una flecha saliente de transición. Puede tener un rótulo de transición (condición de guarda, evento disparador y acción). La Figura 13.189 muestra una unión explícita de estados procedentes de un estado compuesto concurrente.

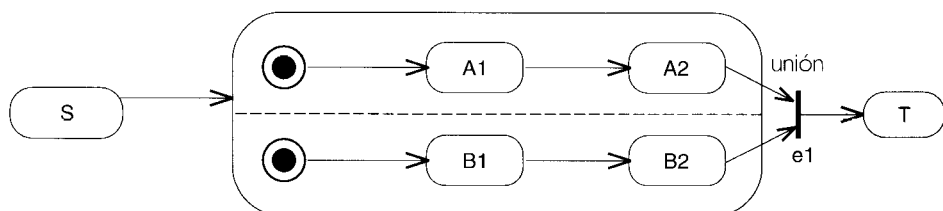


Figura 13.189 Una unión

Discusión

Véase reunificación.

use

Palabra clave para la dependencia de uso en la notación.

uso

Dependencia en la que un elemento (el cliente) requiere la presencia de otro elemento (el proveedor) para su correcto funcionamiento o implementación —generalmente es por razones de implementación.

Véase también colaboración, dependencia, enlace transitorio.

Semántica

Una dependencia de uso es una situación en la cual un elemento requiere la presencia de otro elemento para su correcta implementación o funcionamiento. Todos los elementos tienen que existir en el mismo nivel de significado —esto es, no implican un desplazamiento del nivel de abstracción o de realización (tal como una correspondencia entre una clase del nivel de análisis y una clase de nivel de implementación)—. Con cierta frecuencia las dependencias de uso implican elementos del nivel de implementación, tales como un archivo *include* de C++, lo que implica consecuencias para el compilador. Se puede estereotipar aún más el uso para indicar la naturaleza exacta de la dependencia, tal como llamar a una operación de otra clase o instanciar un objeto de otra clase.

Notación

El uso se denota mediante una flecha discontinua (dependencia) que tiene la palabra clave «**use**». La cabeza de flecha está en el elemento proveedor (independiente) y la cola se encuentra en el elemento cliente (dependiente).

Discusión

Un uso suele corresponder a un enlace transitorio, esto es, una conexión entre instancias de clases que no tiene sentido o no está presente en todo momento, sino sólo en algún contexto, tal como la ejecución de un procedimiento de subrutina. La estructura de dependencia no modela toda la información de esta situación, sino únicamente el hecho de su existencia. La estructura de colaboración proporciona la posibilidad estas relaciones con todo detalle.

Elementos estándar

call, create, instantiate, send.

uso de la tipografía

El texto puede ser distinguido con el uso de diversas tipografías y de otros marcadores gráficos.

Véase también marcador gráfico.

Discusión

Las letras cursivas se utilizan para indicar una clase abstracta, un atributo, o una operación. Otras distinciones de la tipografía están sobre todo para destacar o distinguir las partes de la notación. Se recomienda que los nombres de clasificadores y de asociaciones se muestren en negrillas y elementos subsidiarios, tales como atributos, operaciones, nombre de rol, etcétera, se muestre en tipo normal. Los nombres de compartimentos se deben mostrar en una tipografía distintiva, tal como una negrita pequeña, pero la elección se deja a la herramienta de edición. Una herramienta es también libre de utilizar las distinciones de la tipografía para destacar elementos seleccionados, para distinguir palabras y palabras claves reservadas, y para codificar características seleccionadas de un elemento, o puede permitir el uso de tales distinciones bajo control del usuario. Consideraciones similares se aplican al color, aunque su uso debe ser opcional porque muchas personas no distinguen el color.

Todos los usos mencionados son extensiones convenientes a la notación canónica descrita en este libro, la cual es suficiente para representar cualquier modelo.

utilidad

Un estereotipo de clase que agrupa variables y procedimientos globales en forma de una declaración de clase. Los atributos y operaciones de la utilidad pasan a ser variables y procedimientos globales, respectivamente. Una utilidad no es una estructura fundamental para el modelado, sino algo cómodo para la programación. No posee instancias.

Semántica

Los atributos de la utilidad son variables globales y las operaciones de la utilidad son operaciones globales. Las utilidades son innecesarias para la programación orientada a objetos, por cuanto los atributos y operaciones globales se pueden modelar mejor como miembros con alcance de clase.

Esta estructura se proporciona por compatibilidad con los lenguajes no orientados a objetos, tales como C.

Notación

Las utilidades se representan mediante un símbolo de clase con la palabra clave «**utility**» por encima de la cadena de nombre de clase. Los atributos y operaciones representan miembros globales. En este símbolo no se pueden declarar miembros con alcance de clase.

valor

Véase valor dato.

valor dato

Una instancia de un tipo de dato, un valor sin identidad.

Véase también tipo de dato, objeto.

Semántica

Un valor dato es un miembro de un dominio matemático —un valor puro—. Dos valores dato con la misma representación son indistinguibles; los valores dato no tienen ninguna identidad. Los valores dato son pasados por valor en un lenguaje de programación. No tiene ningún sentido pasarlos por referencia. No tiene sentido hablar de cambiar un valor dato; su valor está fijado permanentemente. De hecho, él es su valor. Cuando uno habla de cambiar un valor de un dato, esto significa generalmente cambiar una variable que contiene un valor de modo que contenga un nuevo valor. Pero los valores dato son invariables.

valor etiquetado

Pareja formada por un marcador y un valor, que se asocia a un elemento para almacenar alguna información. *Véase también* restricción, estereotipo. *Véase* el Capítulo 14, Elementos Estándar, para consultar una lista de marcadores predefinidos.

Semántica

Un valor etiquetado es una pareja valor-selector que se asocia a cualquier elemento (incluyendo elementos del modelo y elementos de presentación) para aportar distintos tipos de información, que suelen ser secundarios para la semántica del elemento pero posiblemente importantes para el esfuerzo global de modelado. El selector se denomina marcador; se trata de un valor de cadena. Cada marcador representa una propiedad que puede ser aplicable a una clase de elemento o a muchas clases. El nombre del marcador sólo puede aparecer una vez en cualquier elemento del modelo. El valor puede ser de distintos tipos, pero se codifica como una cadena. La interpretación del valor es una convención entre el creador del modelo y la herramienta de modelado. Se supone que los valores etiquetados se van a implementar como tablas de búsqueda indexadas por marcadores para tener un acceso eficiente.

Los valores etiquetados representan una información arbitraria expresada en forma de texto y que se suele utilizar para almacenar información relativa a la administración del proyecto, tal como el autor de un elemento, el estado de comprobación o la importancia del elemento para el sistema final (los marcadores podrían ser **autor**, **estado** e **importancia**).

Los valores etiquetados representan una modesta extensión de los metaatributos de las metaclases de UML. No es un mecanismo de extensión completamente general pero se puede utilizar para añadir información a las clases existentes del metamodelo para bien de las herramientas finales, tales como generadores de código, creadores de informes y simuladores. Para evitar la confusión, los marcadores deben ser distintos de los metaatributos existentes de los elementos del modelo al que se aplican. Esta comprobación se puede facilitar mediante una herramienta de modelado.

Ciertos marcadores están predefinidos en el UML; otros pueden ser definidos por el usuario. Los valores etiquetados son un mecanismo de extensión que permite asociar información arbitraria a los modelos.

Notación

Todos los valores etiquetados se muestran en la forma:

`marcador = valor`

en donde `marcador` es el nombre del marcador y `valor` es un valor literal. Los valores etiquetados se pueden incluir junto con otras palabras clave de propiedades en una lista de propiedades separadas por comas y encerrada entre llaves.

Se puede declarar una palabra clave de tal modo que haga las veces de un marcador con un cierto valor. En tal caso, la palabra clave se puede emplear en solitario. La ausencia del marcador se trata como equivalente a uno de los demás valores válidos del marcador.

`marcador`

Ejemplo

`{autor = Héctor, estado = comprobado, requisito = 3.563.2a, suprimir}`

Discusión

Casi todos los programas de edición de modelos ofrecen capacidades básicas para definir, visualizar y buscar valores etiquetados como cadenas, pero no los utilizan para extender la semántica de UML. Sin embargo, las herramientas finales, tales como los generadores de código, creadores de informes y cosas similares, pueden leer los valores etiquetados para alterar su semántica de manera flexible. Observe que las listas de valores etiquetados son una idea muy antigua; por ejemplo, las listas de propiedades del lenguaje Lisp.

Los valores etiquetados son una forma de asociar a los modelos información no semántica de administración y seguimiento del proyecto. Por ejemplo, el marcador `autor` podría contener el autor del elemento y el marcador **estado** podría contener el estado de desarrollo, tal como **incompleto**, **probado**, **defectuoso** y **completo**.

Además, los valores etiquetados son una forma de asociar condiciones dependientes del lenguaje de implementación a los modelos de UML sin incorporar a UML los detalles del lenguaje. Los indicadores propios de un generador de código, las pistas y los pragmas se pueden codificar como valores etiquetados sin afectar al modelo subyacente. Se pueden tener múl-

tiples conjuntos de marcadores para diferentes lenguajes en un mismo modelo. Ni el editor de modelos ni el analizador semántico tienen necesidad de comprender los marcadores —se pueden tratar como si fuesen cadenas—. Una herramienta final, tal como un generador de código, puede comprender y procesar los valores etiquetados. Por ejemplo, un valor etiquetado podría dar nombre a la clase de contenedor que se emplea para anular la implementación por defecto de una asociación con multiplicidad muchos.

Los valores etiquetados satisfacen la necesidad de muchas clases de información que es preciso asociar a los modelos, pero no están destinados a ser un mecanismo completo de extensión de metamodelos. Los marcadores forman un espacio de nombres plano que es preciso administrar empleando convenciones para evitar conflictos de nombres. No se ha previsto la posibilidad de especificar los tipos que poseen su valores. Tampoco están destinados a crear extensiones semánticas serias del lenguaje de modelado en sí. Los marcadores *vienen a ser* como atributos del metamodelo, pero *no* son atributos del metamodelo ni se han formalizado como tales.

El uso de marcadores, tal como el uso de procedimientos, en las bibliotecas de los lenguajes de programación, puede requerir un cierto período de evolución durante el cual puede haber conflictos entre los programadores. Con el tiempo, se desarrollarán estándares de utilización. UML no incluye un “registro” de marcadores ni ofrece la esperanza de que los primeros usuarios de los marcadores puedan “reservarlos” para evitar que se les den otros usos en el futuro.

valor inicial

Se trata de una expresión que especifica el valor que contiene un atributo de un objeto inmediatamente después de ser inicializado.

Véase también valor por defecto, iniciación.

Semántica

Un valor inicial es una expresión asociada a un atributo. La expresión es una cadena de texto que también denota un lenguaje para interpretar la expresión. Cuando se instancia un objeto que contiene ese atributo, se evalúa la expresión según el lenguaje indicado y el valor actual del sistema. El resultado de la evaluación se emplea para iniciar el valor del atributo en el nuevo objeto.

El valor inicial es opcional. Si está ausente, entonces la declaración del atributo no especifica el valor que contendrá en un objeto nuevo (pero quizá otra parte del modelo total proporcione esa información).

Observe que un procedimiento explícito de inicialización para un objeto (un constructor, por ejemplo) puede anular la expresión del valor inicial y reemplazar el valor del atributo.

El valor inicial de un atributo cuyo alcance sea una clase se empleará para iniciarlo al comenzar la ejecución. UML no especifica el orden relativo de inicialización de atributos que posean distintos alcances de clase.

valor no especificado

Valor que todavía no ha sido especificado por el creador del modelo.

Discusión

Un valor no especificado no es un valor en estado correcto y no puede aparecer en un modelo complejo salvo para aquellas propiedades cuyo valor es innecesario o irrelevante. Por ejemplo, la multiplicidad no puede ser desconocida; tiene que tener algún valor. La falta de todo conocimiento es equivalente a una multiplicidad de muchos. La semántica de UML, por tanto, no permite ni trata la ausencia de valores o los valores no especificados.

Hay otro sentido en que el término “no especificado” es una parte útil de un modelo inacabado. Tiene el significado siguiente: “Todavía no he pensado en este valor y he tomado nota de él, así que lo recordaré posteriormente para darle un valor”. Se trata de una indicación explícita de que el modelo está incompleto. Esto es una capacidad útil que muy bien podrían admitir las herramientas. Por su propia naturaleza, tal valor no puede aparecer en un modelo terminado y no tiene sentido definir su semántica. Una herramienta puede proporcionar automáticamente uno por defecto para los valores no especificados cuando se necesite un valor —por ejemplo, en la generación de código— pero esto no pasa de ser una cuestión de comodidad y no forma parte de la semántica. Los valores no especificados van más allá de la semántica de UML.

Análogamente, el valor por defecto de una propiedad carece de significado semántico. Las propiedades tienen un valor, simplemente. Una herramienta puede proporcionar automáticamente valores para las propiedades de los elementos recién creados. Ahora bien, esto no es más que una cierta comodidad operativa dentro de la herramienta; no forma parte de la semántica de UML. Los modelos semánticamente completos de UML no tienen valores por defecto ni valores no especificados; tienen valores, simplemente.

valor por defecto

Un valor proporcionado automáticamente como parte de un cierto lenguaje de programación o de una determinada herramienta. Los valores por defecto para las propiedades de los elementos no son parte de la semántica de UML y no aparecen en modelos.

Véase también valor inicial, parámetro, valor no especificado.

variabilidad

Propiedad que indica si puede cambiar el valor de un atributo o de un enlace.

Semántica

Esta propiedad se puede poner en un extremo de asociación o en un atributo. Si se aplica a una clase, significa que todos los atributos y enlaces de la misma la verifican (por ejemplo, un va-

lor *frozen* significa que el valor de un objeto de la clase no se puede modificar después de la inicialización). La variabilidad se representa mediante una palabra clave en una lista de propiedades con alguno de los siguientes valores:

changeable

Los valores de los atributos pueden cambiar libremente, incluyendo la adición y eliminación de valores si lo permite la multiplicidad. Los enlaces pueden cambiar libremente, y ser añadidos y eliminados respetando las restricciones y la multiplicidad. Éste es el valor por defecto si no se especifica otro.

frozen

Los valores de los atributos no pueden cambiar una vez inicializados, ni pueden añadirse o eliminarse valores. No se pueden añadir enlaces ni modificar los existentes tras la inicialización del objeto del extremo opuesto de la asociación (esto es, el extremo contrario a aquel que tiene el valor congelado). Sin embargo, cuando se crea un objeto en el extremo opuesto se pueden añadir enlaces al extremo congelado como parte de la inicialización.

addOnly

Se pueden añadir valores de atributos adicionales si la multiplicidad no es un número fijo o ya tiene el valor máximo. Tras la creación, no se pueden borrar ni modificar los valores mientras vive el objeto. Se pueden añadir nuevos enlaces pero no se pueden eliminar ni modificar después de su creación. Si se destruye un objeto participante, los objetos que lo contienen son borrados, a pesar de su status **addOnly**.

vértice

Origen o destino de una transición en una máquina de estados. Un vértice puede ser bien un estado o un pseudoestado.

visibilidad

Una enumeración cuyo valor (**pública**, **protegida** o **privada**) denota si el elemento del modelo al que alude se puede o no ver fuera del espacio de nombres que lo contiene.

Véase también acceso para una discusión de las reglas de visibilidad en su aplicación a referencias entre paquetes.

Semántica

La visibilidad declara la capacidad de un elemento de modelado para hacer referencia a un elemento que se encuentra en un espacio de nombres distinto de aquel al que pertenece el elemento que hace la referencia. La visibilidad es parte de la relación existente entre un elemento y el contenedor que lo contiene. El contenedor puede ser un paquete, clase o algún otro espacio de nombres. Existen tres visibilidades predefinidas.

public	Todo elemento que pueda ver el contenedor puede ver también el elemento indicado.
protected	Sólo pueden ver el elemento indicado los elementos pertenecientes a su contenedor o a un descendiente de su contenedor.
private	Sólo pueden ver el elemento indicado los elementos pertenecientes a ese mismo contenedor. Los demás elementos, incluyendo los pertenecientes a descendientes del contenedor, no pueden hacer referencia a él ni usarlo en modo alguno.

Se podrían definir otras clases de visibilidad para algunos lenguajes de programación, tales como la visibilidad *de implementación* propia de C++ (en realidad, todas las formas de visibilidad que no sea pública dependen del lenguaje). El uso de opciones adicionales tiene que hacerse por convenio entre el usuario y las posibles herramientas de modelado y generadores de código.

Notación

La visibilidad se puede mostrar mediante una palabra reservada de propiedad o bien mediante un signo de puntuación situado delante del nombre de algún elemento del modelo.

public	+
protected	#
private	—

Se puede suprimir el marcador de visibilidad. La ausencia de un marcador de visibilidad indica que no se muestra la visibilidad, no que sea pública o no esté definida. Las herramientas deberían asignar visibilidades a los nuevos elementos, aun cuando no se muestre la visibilidad. El marcador de visibilidad es una notación taquigráfica de la cadena completa de especificación de la propiedad de visibilidad.

También se puede especificar la visibilidad mediante palabras clave (**public**, **protected**, **private**). Esta forma suele emplearse como elemento en medio de una lista, aplicándose a todo un bloque de atributos u otros elementos de la lista.

Toda opción de visibilidad que sea propia de un lenguaje o definida por el usuario deberá ser especificada mediante una cadena de propiedad o bien mediante alguna convención propia de una herramienta.

Clases. En las clases el marcador de visibilidad se sitúa en una lista de elementos, tales como atributos y operaciones. Muestra si otras clases pueden o no acceder a esos elementos.

Asociaciones. En una asociación, el marcador de visibilidad se sitúa en el nombre de rol de la clase destino (el extremo al que se accedería empleando esa especificación de visibilidad). Muestra si la clase del extremo opuesto puede o no recorrer la asociación en la dirección del extremo que tiene asociado el marcador de visibilidad.

Paquetes. En un paquete, el marcador de visibilidad se sitúa en elementos contenidos directamente dentro del paquete, tales como clases, asociaciones y paquetes anidados. Muestra si otro paquete que acceda al primero o lo importe puede o no ver los elementos.

vista

Proyección de un modelo, que se ve desde una cierta perspectiva o punto de observación y emite aquellas entidades que no son relevantes para esa perspectiva. Aquí, la palabra no se usa para denotar un elemento de presentación. Se refiere más bien a las proyecciones tanto en el modelo semántico como en la notación visual.

vista de actividades

Aspecto del sistema que tiene que ver con la especificación del comportamiento como actividades conectadas por flujos de control. Esta vista contiene grafos de actividades y se representa mediante diagramas de actividades. De forma un tanto libre se agrupa con otras vistas que tienen que ver con el comportamiento para formar la vista dinámica.

Véase grafo de actividades.

vista de administración del modelo

Dícese de aquel aspecto de un modelo que trata de la organización del modelo en sí en partes estructuradas, a saber, paquetes, subsistemas y modelos. La vista de administración del modelo se considera a veces parte de la vista estática y suele combinarse con la vista estática en los diagramas de clases.

vista de casos de uso

Aspecto del sistema dedicado a la especificación de comportamiento en términos de casos de uso. Un modelo de casos de uso es un modelo que se centra en esta visualización. La vista de casos de uso es parte del conjunto de conceptos de modelado que están relativamente agrupados como vista dinámica.

vista de comportamiento

Vista del modelo que pone de relieve el comportamiento de las instancias existentes en un sistema, incluyendo sus métodos, colaboraciones e historias de estados.

vista de despliegue

Una vista que muestra los nodos en un sistema distribuido, los componentes que se almacenan en cada nodo, y los objetos que se almacenan en componentes y nodos.

Véase despliegue, diagrama de despliegue.